

TUGAS BESAR II
PEMBELAJARAN MESIN IF3270
CONVOLUTIONAL NEURAL NETWORKS AND RECURRENT
NEURAL NETWORKS



Kelompok 53
AnakRudyKusdiantara

Beni Lesmana 10122043
Albi Arrizkya Putra 10122062
Timothy Niels Ruslim 10123053

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I: DESKRIPSI PERSOALAN.....	4
1.1. Latar Belakang.....	4
1.2. Tujuan Tugas.....	5
1.3. Deskripsi Dataset.....	5
BAB II: PEMBAHASAN TEORI.....	6
2.1. Convolutional Neural Networks.....	6
2.1.1. Convolution Stage.....	6
2.1.2. Detector Stage.....	7
2.1.3. Pooling Stage.....	7
2.2. Recurrent Neural Networks.....	8
2.2.1. Recurrent Layers.....	8
2.2.2. Long Short-Term Memory.....	9
2.3. Encoder-Decoders.....	10
2.3.1. Context Vector.....	10
2.3.2. Show and Tell.....	11
BAB III: PENJELASAN IMPLEMENTASI.....	13
3.1. Tensor.....	13
3.2. Komponen Kalkulasi.....	16
3.2.1. Fungsi Aktivasi.....	16
3.2.2. Inisialisasi Bobot.....	17
3.2.3. Fungsi Loss.....	18
3.2.4. Kelas Optimizer.....	19
3.3. Implementasi Layer.....	21
3.3.1. Layer Input.....	22
3.3.2. Layer Dense.....	23
3.3.3. Layer Konvolusi.....	24
3.3.4. Layer Pooling.....	25
3.3.5. Layer Rekuren.....	27
3.3.6. Layer Embedding.....	31
3.3.7. Layer RMSNorm.....	32

3.4. Kelas Model.....	33
3.5. Fungsi Utilitas.....	34
3.5.1. Utilitas Gambar.....	35
3.5.2. Utilitas Teks.....	35
3.5.3. Keras Weight Loader.....	36
BAB IV: HASIL PENGUJIAN DAN ANALISIS.....	37
4.1. CNN: Image Classification.....	37
4.1.1. Pengaruh Jumlah Layer Konvolusi.....	40
4.1.2. Pengaruh Banyak Filter.....	41
4.1.3. Pengaruh Ukuran Filter.....	42
4.1.4. Pengaruh Jenis Pooling Layer.....	43
4.1.5. Pengaruh Parameter Sharing.....	44
4.1.6. Perbandingan Keras vs Scratch.....	45
4.1.7. Visualisasi Grad-CAM.....	46
4.2. RNN dan LSTM: Image Captioning.....	48
4.2.1. Perbandingan Jumlah Layer: RNN.....	50
4.2.2. Perbandingan Jumlah Layer: LSTM.....	50
4.2.3. Perbandingan Ukuran Hidden State: RNN.....	51
4.2.4. Perbandingan Ukuran Hidden State: LSTM.....	52
4.2.5. Perbandingan RNN vs LSTM.....	52
4.2.6. Analisis Kualitatif Image Captioning.....	53
4.2.7. Perbandingan Keras vs Scratch.....	56
4.2.8. Pengaruh Panjang Maksimum Caption.....	57
4.2.9. Perbandingan Init-Inject.....	57
4.2.10. Perbandingan Beam Search.....	58
BAB V: KESIMPULAN DAN SARAN.....	60
4.1. Kesimpulan.....	60
4.2. Saran.....	60
DAFTAR PUSTAKA.....	61
LAMPIRAN.....	62

BAB I

DESKRIPSI PERSOALAN

1.1. Latar Belakang

Bidang *deep learning* bermula dari *Feed Forward Neural Network* (FFNN), yang hanya terdiri dari banyak lapisan perceptron, sehingga dapat dilatih untuk memodelkan fungsi yang kompleks secara akurat. Walaupun efektif, FFNN memiliki jumlah parameter yang sangat banyak, sehingga membuatnya mahal secara komputasional, dari segi waktu latih yang dibutuhkan maupun data latih yang cukup untuk melatih seluruh parameter tersebut. Oleh karena itu, kita bisa memasukkan *inductive bias* pada arsitektur model yang dibangun, untuk menghasilkan model yang dapat dikhususkan pada jenis data yang dimiliki, dengan harapan mengurangi jumlah parameter sembari mempertahankan kinerja.

Pertama, ada *Convolutional Neural Network* (CNN) yang dikhususkan untuk fitur spasial, seperti gambar. Dia mengasumsikan *parameter sharing* dan *sparse connectivity*. Asumsi tersebut menghasilkan layer yang berupa konvolusi dari fitur dengan suatu kernel untuk mendeteksi pola lokal. Lalu, ada juga operasi *pooling* untuk mereduksi dimensi. Dengan itu, CNN dapat secara drastis mengurangi jumlah parameter yang perlu dilati, tetapi tetap bisa mempertahankan, bahkan melampaui, kemampuan FFNN untuk data spasial.

Kedua, ada juga *Recurrent Neural Network* (RNN) yang dikhususkan untuk fitur sekuensial, seperti time series atau teks. Dia menerapkan *parameter sharing through time*. Yakni, layer yang diimplementasikan mempertahankan hidden state dari timestep sebelumnya untuk menghitung hidden layer pada timestep berikutnya. Adapun suatu jenis RNN yaitu *Long Short-Term Memory* (LSTM) yang menambahkan adanya juga cell state yang dipertahankan sepanjang timestep. Maka, kedua arsitektur ini memang dirancang untuk menangani data sekuensial dengan baik.

Untuk permasalahan multimodal (banyak jenis data), semua arsitektur ini (FFNN, CNN, maupun RNN) dapat digabungkan untuk menghasilkan model yang lebih optimal. Contohnya, arsitektur *Encoder-Decoder* dapat mengintegrasikan suatu model pertama, disebut *encoder*, untuk menghasilkan context vector yang melambangkan representasi fitur pada latent space, lalu menggunakan model kedua, disebut *decoder*, untuk menghasilkan outputnya (termasuk sekuens). Selain arsitektur yang sudah dibahas ini, banyak lagi model *deep learning* yang dapat dirancang berdasarkan *inductive bias* yang diasumsikan.

1.2. Tujuan Tugas

Dengan munculnya banyak jenis arsitektur yang telah dibahas di atas, Tugas Besar II ini bertujuan agar kami dapat mengimplementasikan *Convolutional Neural Network* (CNN) dan *Recurrent Neural Network* (RNN) secara mandiri. Pada tugas ini, kami ditugaskan untuk mengimplementasikan modul forward propagation FFNN (yakni dense), CNN (seperti Conv2D dan LocallyConnected2D), dan RNN (seperti SimpleRNN dan LSTM) sepenuhnya dari *scratch*.

Selain itu, kami akan menerapkan implementasi tersebut pada dua masalah penting. Pertama, diselesaikan masalah *image classification* melalui arsitektur CNN murni secara standar. Kedua, dieksplorasi pipeline *image captioning* melalui arsitektur Encoder-Decoder yang menggabungkan CNN dan LSTM. Tugas ini juga menjadi latihan fondasional untuk mengerti sistem *deep learning* lebih advanced lainnya, seperti LLM multimodal dan speech recognition.

1.3. Deskripsi Dataset

Untuk masalah *image classification*, digunakan dataset [Intel Image Classification](#) yang mengandung ~25.000 gambar pemandangan alam berukuran 150 x 150 piksel. Terdapat 6 kelas gambar: buildings, forest, glacier, mountain, sea, and street. Lalu, dataset-nya sendiri sudah mendukung *data split* secara default: train set (~14,000), test set (~3,000), dan prediction set (~7,000). Oleh karena itu, masalah utamanya adalah melatih CNN untuk secara akurat mengkategorikan gambar pemandangan alam tersebut meskipun terdapat banyak variasi pencahayaan, sudut pandang, dan tekstur di dunia nyata.

Berikutnya, masalah *image captioning* menggunakan dataset [Flickr 8k](#) yang mengandung 8.000 gambar acak, tetapi dilengkapi secara rinci dalam file csv dengan *captions* yang mendeskripsikan objek, adegan, maupun aksi pada gambar. Dirancang untuk pembuatan caption gambar, alat ini menguji kemampuan model untuk menghasilkan deskripsi tekstual semirip mungkin dengan manusia yang melakukannya secara visual. Tantangannya terletak pada menjembatani data visual dan bahasa, menangkap konteks, dan menghasilkan keterangan yang koheren dan manusia.

BAB II

PEMBAHASAN TEORI

2.1. Convolutional Neural Networks

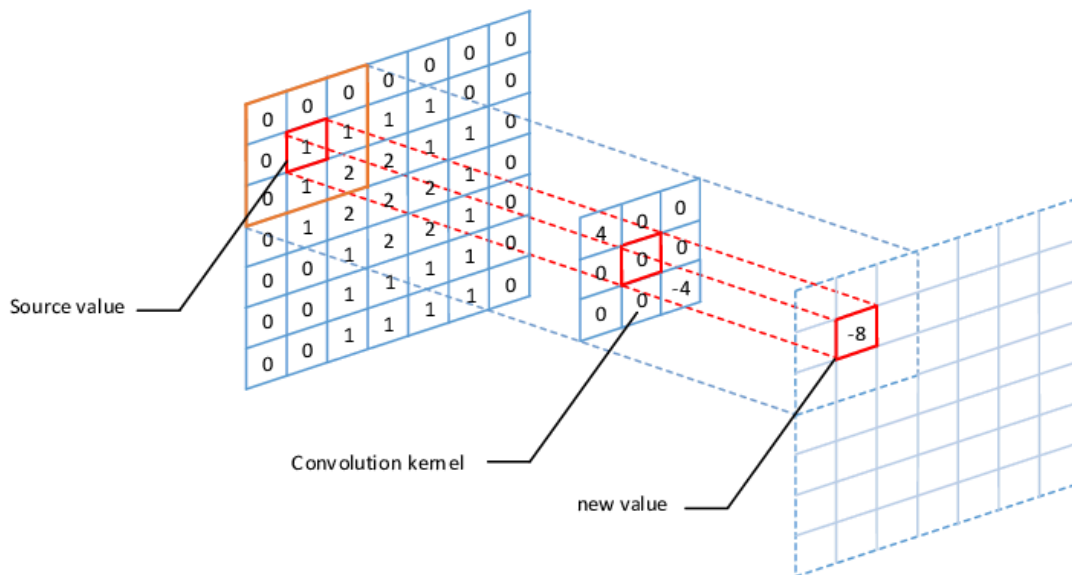
Convolutional Neural Networks (CNN) adalah arsitektur *deep learning* yang secara spesial didesain untuk data spasial, khususnya grid seperti gambar. Dengan mengasumsikan *inductive bias* invariansi translasi dan lokalitas, diperoleh dua hal yang membedakan CNN dari FFNN: *parameter sharing* dan *sparse connectivity*. Terinspirasi dari otak manusia yang tidak seluruhnya menyala saat berpikir (melainkan secara lokal), CNN dibuat untuk mengatasi masalah banyaknya parameter dan ketidaktahuan informasi spasial pada FFNN. Dengan itu, terdapat tiga komponen utama pada *forward pass* CNN: konvolusi, deteksi, dan pooling.

2.1.1. Convolution Stage

Lapisan konvolusi menerapkan suatu kernel atau filter pada fitur melalui konvolusi diskrit. Secara eksplisit, misalkan dimiliki masukan fitur 2D dengan C channel berupa array $X \in R^{W \times H \times C}$ dan kernel yang sesuai $K \in R^{F \times F \times C \times D}$. Maka, hasil peta fitur $Y \in R^{W' \times H' \times D}$ dengan konvolusi biasa di posisi (i, j) diberikan oleh

$$Y_{i,j,k} = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} \sum_{c=0}^{C-1} X_{Si+m, Sj+n, c} K_{m,n,c,k} + b_k$$

dengan $b \in R^D$ adalah bias. Berikut gambar visualisasi proses konvolusi 2D ini.



Selebihnya, perhatikan bahwa konvolusi pasti akan membuat hasil peta fitur yang lebih kecil daripada masukannya (karena agregasi informasi lokal menjadi terpusat di satu sel). Rumus dimensi baru dari peta fitur diberikan oleh

$$H' = \lfloor \frac{H+2P-F}{S} \rfloor + 1$$

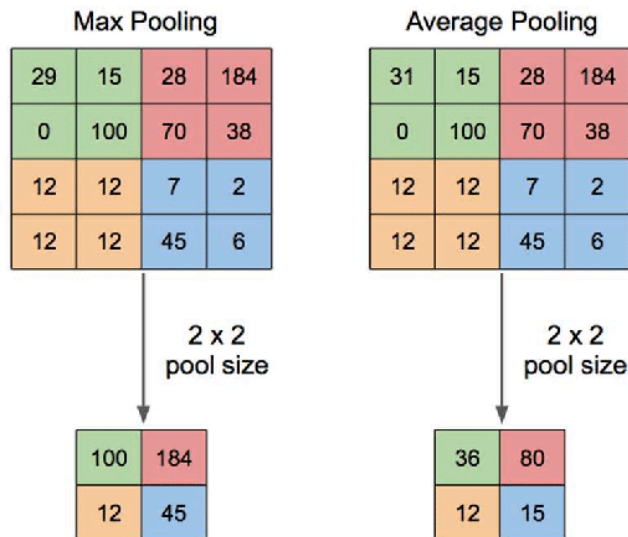
dan hal yang serupa dengan W' . Di sini, kita punya P adalah *padding size* (pembesaran gambar dengan piksel kosong sebelum konvolusi) dan S adalah *stride* (seberapa jauh filter meloncat saat konvolusi) yang digunakan.

2.1.2. Detector Stage

Lapisan deteksi adalah penerapan suatu fungsi aktivasi non-linear pada hasil konvolusi. Di CNN, fungsi aktivasi paling umum adalah $ReLU(z) = \max(0, z)$ yang diterapkan secara *element-wise*. Aktivasi dinamakan “detektor” karena berperan sebagai *threshold* untuk nilai-nilai hasil konvolusi yang diinginkan, sehingga dapat menguatkan sinyal yang dihasilkan.

2.1.3. Pooling Stage

Lapisan *pooling* melakukan reduksi dimensi pada peta fitur *post-activation*. Contoh yang paling umum adalah *max-pooling* (mengambil maksimum) dan *average-pooling* (mengambil rata-rata) menggunakan filter berjalan seperti saat konvolusi (tetapi tanpa parameter yang perlu dipelajari).



Sebagai contoh, *max-pooling* dari suatu peta fitur Z dengan jendela sebesar $F \times F$ menghasilkan

$$P_{i,j,k} = \max_{0 \leq m,n < F} Z_{si+m,sj+n,k}$$

dengan s adalah stride dari operasi pooling. Seperti pada konvolusi, rumus

$$H' = \lfloor \frac{H+2P-F}{s} \rfloor + 1$$

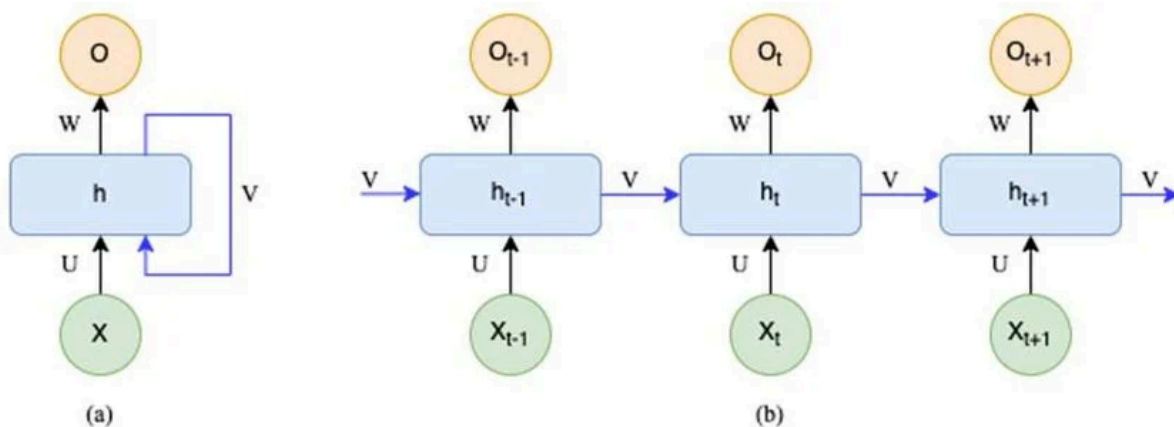
tetap berlaku. Dengan melakukan *pooling*, walaupun konvolusi bersifat lokal, informasi dari seluruh gambar dapat dikumpulkan pada sel-sel pusat. Reduksi dimensi ini membantu secara komputasional, tetapi juga dapat mengurangi *overfitting*.

2.2. Recurrent Neural Networks

Recurrent Neural Network (RNN) adalah arsitektur *deep learning* yang secara khusus didesain untuk data sekuensial, seperti time series dan teks. Termotivasi dari masalah forecasting dan pemodelan bahasa, RNN memperhitungkan pentingnya urutan data melalui komponen rekuren. Diterapkan *inductive bias* terhadap struktur sekuensial: bahwa konteks dari “masa lalu” atau “token sebelumnya” yang mempengaruhi pola di masa kini. Maka, RNN dapat menghafal informasi dari masa lalu secara berkala, tidak seperti FFNN yang hanya bisa memodelkan satu time step atau perlu mem-flatten sekuens sehingga melupakan urutan.

2.2.1. Recurrent Layers

RNN menerapkan *parameter sharing through time* dalam layer rekuren. Sebagai terminologi, hasil output dari suatu layer rekuren disebut *hidden state*. Sekarang, selain bobot yang digunakan untuk menghitung pengaruh layer sebelumnya, layer rekuren juga memiliki bobot untuk merepresentasikan pengaruh nilai hidden state dari time step sebelumnya pada layer rekuren yang sama. Berikut gambar *folded* (kiri) dan *unfolded* (kanan) dari suatu lapisan rekuren. Terlihat bahwa input berupa suatu sekuens, dan layer rekuren dapat memprosesnya karena adanya bobot antar time step yang di-*share*.



Secara konkrit, misalkan x_t adalah hasil dari layer sebelumnya di time step sekarang dan h_{t-1} adalah nilai hidden state yang dihasilkan oleh layer rekuren pada time step sebelumnya. Maka, layer rekuren akan mengeluarkan

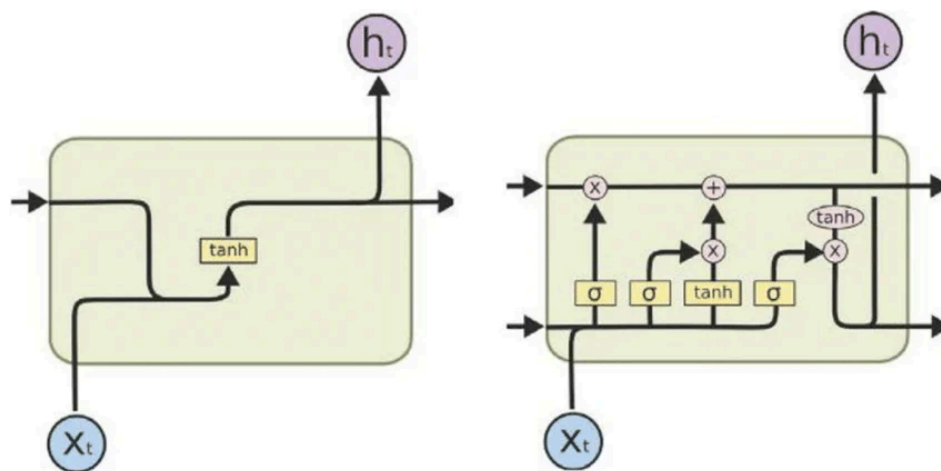
$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

pada time step sekarang, dengan b_h adalah bias dan f adalah fungsi aktivasi yang digunakan. Dalam layer rekuren sederhana, digunakan $f(z) = \tanh(z)$ secara konvensional.

Walaupun rekayasa layer rekuren sudah bisa memperhitungkan pengaruh konteks sekuensial pada hasil output, RNN vanilla ini mengalami masalah kesulitan memodelkan *long-term dependency*. Alasannya adalah bahwa, tergantung pemilihan turunan fungsi aktivasi f dan juga nilai eigen dari matriks bobot yang dipelajari, nilai gradien yang dihasilkan saat melakukan pelatihan dapat meledak terlalu besar (*exploding gradients*) atau menghilang menjadi sangat kecil (*vanishing gradient*), sehingga sulit melakukan update pada bobot-bobot time step awal melalui *backpropagation through time*.

2.2.2. Long Short-Term Memory

Dalam upaya untuk menyelesaikan masalah *long-term dependency* tersebut, terlahir arsitektur *Long Short-Term Memory* (LSTM). Selain hidden state h_t pada setiap time step, LSTM menghitung suatu *cell state* c_t yang seperti “ingatan” untuk informasi dari time step sebelumnya secara jangka panjang.



Perhitungan hidden dan cell state adalah dengan tiga jenis gate (forget gate f_t yang menghitung berapa informasi dari cell state untuk dilupakan, input gate i_t yang menghitung berapa informasi yang harus ditambahkan pada cell state, dan output gate o_t yang menghitung kontribusi

informasi time step ke- t pada hidden state) dan suatu candidate value. Gambar di atas memberikan visualisasi *gate* tersebut, dan perbandingannya dengan suatu sel RNN vanilla. Selanjutnya, rinci kalkulasi adalah sebagai berikut.

$$\begin{aligned}
f_t &= \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \\
i_t &= \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \\
\hat{C}_t &= \sigma(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \\
o_t &= \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \hat{C}_t \\
h_t &= o_{t-1} \odot \tanh(c_t)
\end{aligned}$$

Di atas, σ adalah fungsi sigmoid dan semua b adalah bias. Perhatikan bahwa c_t sendiri hanya berinteraksi dengan informasi pada sel ke- t melalui penjumlahan dan perkalian *element-wise*, sehingga tidak mudah terganggu oleh aliran antar time-step. Jadi, ini dapat mengurangi kemungkinan *vanishing* atau *exploding gradients*. Di sisi lain, LSTM juga memiliki empat kali lebih banyak parameter daripada RNN sederhana, sehingga lebih mahal secara komputasional.

2.3. Encoder-Decoders

Dengan banyaknya jenis model yang telah dikembangkan, dari FFNN, CNN, dan RNN, diperlukan suatu cara sistematis untuk mengetahui bagaimana menggunakan mereka secara terpisah maupun bersama. Salah satu arsitektur yang terlahir dari *penggabungan* model-model tersebut adalah *Encoder-Decoder*. Arsitektur ini bagus untuk permasalahan *sequence to sequence* karena membagikan tugas tersebut pada dua model terpisah yang bekerja sama: suatu *encoder* dan *decoder*. Kerja sama tersebut juga membuat *encoder-decoder* cocok untuk data multimodal (banyak jenisnya) seperti di permasalahan *image captioning*.

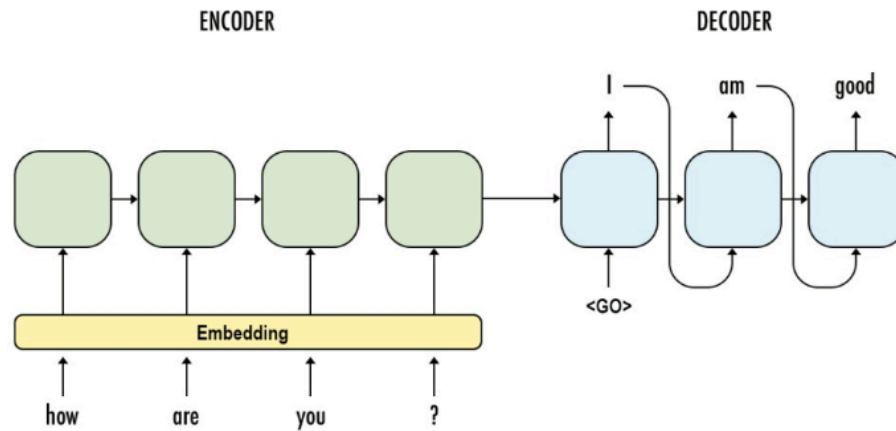
2.3.1. Context Vector

Cara kerja *encoder-decoder* adalah sebagai berikut. Pertama, terdapat model *encoder* (seperti CNN untuk input gambar atau LSTM untuk input sekuens) yang akan menghasilkan suatu hidden state terakhir dari input yang disediakan. Hidden state ini dapat dianggap sebagai representasi data tersebut di suatu *latent space* matematis yang dapat lebih mudah dimengerti oleh komputer.

Hidden state yang dihasilkan tersebut akan dioperkan ke model *decoder* (yang dapat berupa LSTM untuk output teks atau sekuens misalnya). Oleh karena itu, vektor tersebut diberikan istilah *context vector*. Vektor konteks ini di-*inject* salah satu masukan dari *decoder*. Ada beberapa cara untuk melakukan itu.

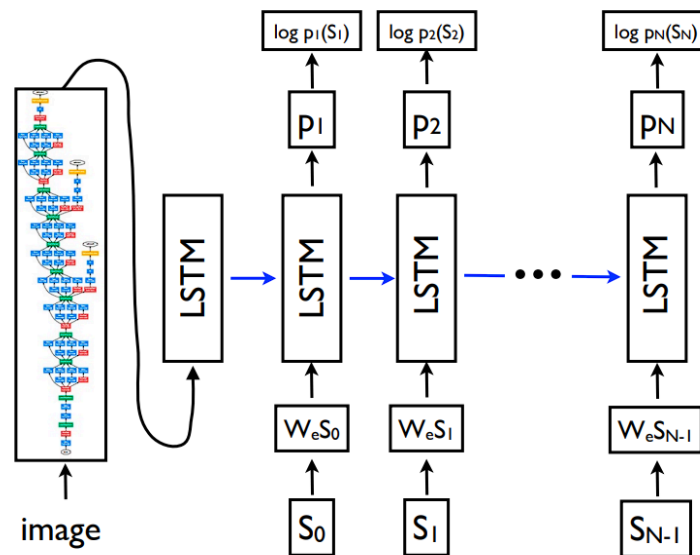
1. *Init-Injection*. Vektor konteks menjadi *hidden state* time step pertama (h_0) di *decoder*.
2. *Pre-Injection*. Vektor konteks menjadi *input token* time step pertama (x_0) di *decoder*.
3. *Post-Injection*. Vektor konteks menjadi *input token* time step terakhir (x_{T+1}) di *decoder*.
4. *Par-Injection*. Vektor konteks dikonkatenasikan dengan semua *input token* di *decoder*.

Di bawah adalah gambar arsitektur *encoder-decoder* dengan metode *init-inject*. Sel-sel hijau adalah *encoder* sedangkan sel-sel biru adalah *decoder*. Di kasus ini, keduanya berupa RNN.



2.3.2. Show and Tell

Salah satu contoh model *encoder-decoder* adalah arsitektur “Show and Tell” oleh Vinyals et al., yang akan kami implementasikan juga pada tugas besar ini. Arsitektur ini berupaya untuk menyelesaikan masalah *image captioning*.



Untuk melakukan itu, digunakan *encoder* berupa *pretrained* CNN (seperti Inception) untuk mendapatkan suatu vektor konteks untuk merepresentasikan data gambar pada *latent space*. Berikutnya, digunakan layer LSTM (dengan aktivasi standar) dan Dense (dengan aktivasi softmax) sebagai *decoder*. Konteks vektor digunakan dengan metode *pre-inject* pada masukan berupa sekuens teks caption. Saat pelatihan, teks input tersebut berdasarkan *teacher forcing*, sedangkan saat inferensi, teks input tersebut berdasarkan mekanisme autoregressive dengan *beam search*.

BAB III

PENJELASAN IMPLEMENTASI

Implementasi package bisa dilihat di folder `src/nn`. Dalam package `nn`, kami akan mengadopsi struktur folder berikut. Ini memastikan modularisasi kode kami tetap terstruktur dan bersih.

```
nn/
├── layers/
│   ├── conv.py
│   ├── dense.py
│   ├── embedding.py
│   ├── layer.py
│   ├── normalization.py
│   ├── pool.py
│   └── recurrent.py
├── activations.py
├── base.py
├── engine.py
├── initializers.py
├── losses.py
├── model.py
└── optimizers.py
```

Penjelasan khususnya akan diterangkan di subbab-subbab berikutnya.

3.1. Tensor

Pertama, dibuat data struktur kustom yaitu `Tensor` yang menggantikan suatu larik NumPy. Ini terlihat di `engine.py`. Kita akan sebut suatu instansi dari kelas ini dengan sebutan *tensor*. Untuk mengimplementasikan tensor, diperlukan fungsi pembantu berikut.

engine.py: Fungsi Global			
Nama	Parameter	Tipe Return	Deskripsi
<code>_unbroadcast</code>	<code>grad, shape</code>	<code>np.ndarray</code>	Membalikan <i>broadcasting</i> yang dilakukan secara default oleh NumPy.
<code>_restore_shape</code>	<code>grad, original_shape, axis</code>	<code>np.ndarray</code>	Mengembalikan dimensi yang hilang dengan menambahkan <i>axes</i> ukuran 1.

Berikutnya, diperlukan juga kelas pembantu berikut sebagai tambahan kelas `Tensor`.

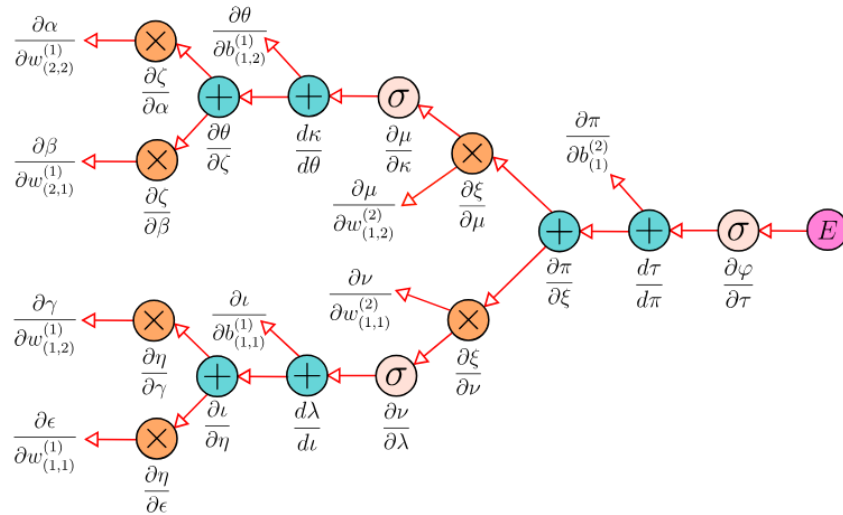
engine.py: Kelas Global			
Nama	Atribut	Metode	Deskripsi
_GradMode	enabled	-	Pengelola status penggunaan fitur <i>auto-differentiation</i> .
no_grad	prev	__enter__, __exit__	Pengelola konteks (<i>toggle</i>) penggunaan fitur <i>auto-differentiation</i> .
Tensor	Data, grad, requires_grad, _children, _backward	__add__, __mul__, __getitem__, ..., transpose, concatenate, backward	Pengganti array NumPy karena bisa menyimpan data sekaligus gradien. Menjadi simpul di graf komputasi <i>auto-differentiation</i> .

Sekarang, dalam implementasinya, suatu tensor tidak hanya menyimpan data yang ditampungnya seperti array NumPy biasa, melainkan juga menyimpan tensor-tensor lain yang mengakibatkan nilai yang disimpannya, nilai turunan loss terhadapnya, dan juga bagaimana mempropagasikan errornya ke tensor lain yang berhubungan.

Kelas Tensor		
Nama	Tipe Data / Return	Deskripsi
Atribut		
data	np.ndarray	Nilai dari suatu tensor.
grad	np.ndarray	Turunan dari suatu tensor.
requires_grad	bool	<i>Flag</i> yang melambangkan apakah tensor termasuk <i>trainable</i> .
_children	tuple	Koleksi tensor yang menjadi anak dari tensor induk pada graf komputasi.
_backward	Callable	Menghitung turunan dari tensor anak dengan aturan rantai.
Metode		
__add__, __radd__	Tensor	Menjumlahkan dua tensor.
__neg__	Tensor	Negatif suatu tensor.

<code>--sub--</code>	Tensor	Selisih dua tensor.
<code>--mul--</code> , <code>--rmul--</code>	Tensor	Perkalian <i>element-wise</i> dua tensor.
<code>--truediv--</code>	Tensor	Pembagian <i>element-wise</i> dua tensor.
<code>--matmul--</code> , <code>--rmatmul--</code>	Tensor	Perkalian matriks dua tensor.
<code>--pow--</code>	Tensor	Perpangkatan <i>element-wise</i> suatu tensor.
<code>--abs--</code>	Tensor	Nilai mutlak <i>element-wise</i> suatu tensor.
<code>--getitem--</code>	Tensor	Mengakses sub-tensor pada indeks tertentu suatu tensor.
<code>sum</code>	Tensor	Menjumlahkan semua nilai tensor terhadap suatu <i>axis</i> .
<code>max</code>	Tensor	Nilai maksimum suatu tensor terhadap suatu <i>axis</i> .
<code>mean</code>	Tensor	Nilai rata-rata suatu tensor terhadap suatu <i>axis</i> .
<code>reshape</code>	Tensor	Menggantikan ukuran tensor.
<code>split</code>	<code>list[Tensor]</code>	Memecah tensor berdasarkan indeks menjadi koleksi tensor.
<code>transpose</code>	Tensor	Transpose dari suatu tensor.
<code>pad</code>	Tensor	Padding (penambahan nilai di luar batas) suatu tensor.
<code>concatenate</code>	Tensor	Penggabungan (konkatenasi) dua tensor.
<code>backward</code>	-	Membangun urutan topologi dari graf komputasi yang berakar di tensor tersebut dan mempropagasikan turunannya ke bawah.

Unsur-unsur ini membuat suatu tensor struktur data yang cocok untuk melakukan *forward* dan *backward pass* sekaligus menggunakan metode *auto-differentiation*. Metode *auto-differentiation* adalah suatu cara menghitung turunan, yakni *backpropagation*, dengan cukup mendefinisikannya pada operasi-operasi elementer pada level tensor-nya sendiri. Ini berarti kita tidak perlu sulit memikirkan cara melakukan *backpropagation* yang sulit untuk setiap jenis layer (misalnya *backpropagation through time*), melainkan cukup mendefinisikan *forward pass* tiap layer dengan operasi-operasi elementer yang *backpropagation*-nya sudah dipikirkan.



Di implementasi tensor kami misalnya, kami selalu melacak tensor-tensor apa saja yang menghasilkan data pada tensor yang sedang ditinjau (yakni di `_children`). Lalu, cukup mendefinisikan bagaimana menghitung turunan tensor-tensor anak tersebut jika sudah diketahui turunan tensor yang ditinjau. Ini akan menghasilkan graf komputasi *auto-differentiation* seperti di atas. Untuk benar-benar melakukan *backpropagation*, berarti cukup memanggil method `backward` pada tensor yang mewakili nilai loss function. Dengan itu, turunannya akan di propagasikan ke seluruh graf komputasi secara berurutan.

3.2. Komponen Kalkulasi

Setelah dibangun *engine* berupa tensor kami, dapat didefinisikan beberapa komponen-komponen utama dari suatu *neural network* menggunakan tensor tersebut. Ini termasuk fungsi aktivasi, metode inisialisasi bobot, fungsi loss, dan metode optimasi.

3.2.1. Fungsi Aktivasi

Fungsi aktivasi adalah suatu fungsi non-linear yang ada di hampir semua layer *neural network*. Jadi, kami implementasikan fungsi-fungsi aktivasi yang umum digunakan secara terpisah agar modular. Berikut adalah rincinya.

activations.py: Fungsi Global			
Nama	Parameter	Type Return	Deskripsi
linear	Tensor	Tensor	$f(z) = z$
relu	Tensor	Tensor	$f(z) = \max(0, z)$

sigmoid	Tensor	Tensor	$f(z) = \frac{1}{1+e^{-z}}$
tanh	Tensor	Tensor	$f(z) = \tanh(z)$
softmax	Tensor	Tensor	$f(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$
swish	Tensor	Tensor	$f(z) = \frac{z}{1+e^{-z}}$
gelu	Tensor	Tensor	$f(z) = z\Phi(z)$

Selebihnya, perhatikan bahwa walaupun *auto-differentiation* memungkinkan kami untuk hanya mendefinisikan *forward pass* dari fungsi aktivasi dengan hanya menggunakan operasi elementer yang telah dibuat, kami juga tetap secara manual mendefinisikan metode `_backward` untuk aktivasi. Alasannya adalah efisiensi, karena beberapa turunan fungsi aktivasi sebenarnya sangat sederhana sehingga lebih cepat menggunakan turunan tersebut daripada satu-satu dari operasi elementer.

3.2.2. Inisialisasi Bobot

Metode inisialisasi bobot adalah cara-cara untuk menginisialisasi nilai parameter yang ada pada suatu layer (seperti matriks bobot pada layer dense, kernel pada layer konvolusi, dan lainnya). Berikut beberapa metode standar pada industri yang kami implementasikan. Semua rumus matematika berlaku secara *element-wise*.

initializers.py: Fungsi Global			
Nama	Parameter	Tipe Return	Deskripsi
<code>_compute_connections</code>	<code>tuple(int)</code>	<code>tuple</code>	Menghitung dimensi masukan dan keluaran, khususnya ini untuk bobot kernel.
<code>zero</code>	<code>tuple(int), np.random.Generator</code>	Tensor	$W = 0$
<code>uniform</code>	<code>tuple(int), float, float, np.random.Generator</code>	Tensor	$W \sim U(\alpha, \beta)$

normal	tuple(int), float, float, np.random.Generator	Tensor	$W \sim N(\mu, \sigma^2)$
xavier	tuple(int), np.random.Generator	Tensor	$W \sim U(-\sqrt{\frac{6}{r+c}}, \sqrt{\frac{6}{r+c}})$
he	tuple(int), np.random.Generator	Tensor	$W \sim N(0, \frac{2}{r})$

Untuk semua layer yang akan kami buat nanti, metode inisialisasi bobot akan secara default menggunakan standar industri (xavier untuk bobot dense, zero untuk bias, he untuk kernel konvolusi, dan lainnya).

3.2.3. Fungsi Loss

Untuk melatih suatu *neural network*, kita akan mencari bobot-bobot yang meminimalkan suatu fungsi loss. Fungsi loss mengukur seberapa berbeda prediksi dari model dengan data asli. Tergantung apakah model adalah untuk klasifikasi atau regresi, juga tergantung modalitas data yang dimiliki, fungsi loss yang digunakan akan berbeda-beda. Di sini, kami implementasikan empat fungsi loss umum berikut.

initializers.py: Fungsi Global			
Nama	Parameter	Tipe Return	Deskripsi
mse	Tensor, Tensor	Tensor	<i>Mean Square Error</i> $\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y)^2$
bce	Tensor, Tensor	Tensor	<i>Binary Cross-Entropy</i> $-\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)]$
cce	Tensor, Tensor	Tensor	<i>Categorical Cross-Entropy</i> $-\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij})$
scce	Tensor, Tensor, int	Tensor	<i>Sparse Categorical Cross-Entropy</i> $-\frac{1}{N_{valid}} \sum_{i \text{ valid}} \log(\hat{y}_{i,valid})$

Di sini, highlight utamanya adalah fungsi loss *Sparse Categorical Cross-Entropy* terakhir, yang akan digunakan untuk implementasi arsitektur CNN dan RNN di tahap berikutnya. Sebenarnya rumusnya sama dengan *Categorical Cross-Entropy* biasa, hanya saja dengan asumsi prediksinya sudah berbentuk kelasnya, bukan *one-hot encoding*-nya.

3.2.4. Kelas Optimizer

Setelah memilih suatu fungsi loss, kita akan mengupdate bobot-bobot yang telah diinisialisasikan berdasarkan nilai loss yang diperoleh berdasarkan suatu prediksi. Khususnya, kita akan menggunakan nilai turunan loss terhadap bobot untuk melakukan update, dengan adanya faktor *learning rate* untuk mengatur sensitivitasnya. Untuk melakukan itu, kami mengimplementasikan suatu kelas optimizer yang menjadi kelas induk.

Kelas Optimizer		
Nama	Type Data / Return	Deskripsi
Atribut		
parameters	list[Tensor]	Semua parameter model (bobot dan bias) yang kelas ini punya akses pada.
alpha	float	Learning rate yang mengatur sensitivitas turunan loss pada update.
clip_value	float	Batas nilai gradien (untuk dipotong saat terlalu besar)
clip_norm	float	Batas norma gradien (untuk dinormalisasikan saat terlalu besar)
params	dict	Hyperparameter untuk metode optimisasi.
Metode		
step	-	Menjalankan satu langkah update bobot berdasarkan aturan yang sesuai.
build	-	Menginisialisasi parameters <i>trainable</i> yang diperoleh dari model.
zero_grad	-	Mengenolkan semua nilai gradien parameters (digunakan sebelum melakukan iterasi <i>backward pass</i> berikut)
_clip_gradients	-	Melakukan pemotongan atau normalisasi

		gradien yang terlalu “besar”.
--	--	-------------------------------

Lalu, metode optimasi *Stochastic Gradient Descent* di SGD dan *Adaptive Moment Estimation* di Adam meng-*inherit* kelas *Optimizer* tersebut. Kelas SGD sendiri menggunakan update bobot sesuai rumus standar

$$\theta_t = \theta_{t-1} - \alpha \frac{dL}{d\theta},$$

sehingga hanya perlu mengubah fungsi *step* (tidak perlu implementasi metode lainnya). Di sisi lainnya, Adam memerlukan hyperparameter tambahan. Detailnya pada tabel berikut.

Kelas Adam		
Nama	Tipe Data / Return	Deskripsi
Atribut		
beta1	float	Tingkat peluruhan eksponensial untuk estimasi momen pertama.
beta2	float	Tingkat peluruhan eksponensial untuk estimasi momen kedua.
epsilon	float	Konstan aditif untuk <i>numerical stability</i> (agar tidak membagi dengan nol)
t	int	Mencatat jumlah epoch (digunakan untuk menghitung koreksi momen)
m	list[np.darray]	List estimasi momen pertama untuk setiap parameter.
v	list[np.darray]	List estimasi momen kedua untuk setiap parameter.
Metode		
step	-	Melangkah satu update bobot berdasarkan rumus Adam. $m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$ $v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$ $\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$ $w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$

build	-	Selain menginisialisasi parameter <i>trainable</i> , juga menginisialisasi list <i>m</i> dan <i>v</i> .
-------	---	---

Dalam implementasi model CNN untuk *image classification* dan RNN untuk *image captioning*, kami akan menggunakan metode Adam.

3.3. Implementasi Layer

Berikutnya, akan dibuat layer-layer dari *neural network*. Untuk mengimplementasikan hal tersebut, kami buat dulu suatu kelas induk besar yang merepresentasikan suatu struktur dalam *neural network* bernama *Module*. Nanti, modelnya sendiri dan layer-layernya akan diimplementasikan sebagai kelas anak dari modul ini.

Kelas Module		
Nama	Tipe Data / Return	Deskripsi
Atribut		
-	-	-
Metode		
tensors	list[<i>Tensor</i>]	Mencari semua parameter yang ada pada seluruh modul.
parameters	list[<i>Tensor</i>]	Mencari semua parameter <i>trainable</i> yang ada pada seluruh modul.
zero_grad	-	Mengenolkan turunan semua parameter <i>trainable</i> .
regularization_loss	<i>Tensor</i>	Untuk menghitung kontribusi regularisasi terhadap loss. Di kelas induk cukup mengembalikan tensor nol.
freeze	-	Membekukan parameter pada modul sehingga menjadi <i>untrainable</i> .
unfreeze	-	Membuat parameter pada modul menjadi <i>trainable</i> lagi.

Berikutnya, suatu layer sendiri menjadi kelas *Layer* yang meng-*inherit* dari *Module* yang bisa dipanggil untuk melakukan *forward pass*. Layer juga melacak ukuran masukan dan keluarannya supaya bisa membangun matriks bobot yang sesuai.

Kelas Layer		
Nama	Tipe Data / Return	Deskripsi
Atribut		
-	-	-
Metode		
forward	Tensor	Di sini belum diimplementasikan, nantinya untuk <i>forward pass</i> .
build	-	Menginisialisasikan ukuran input sehingga bisa membangun parameter yang diperlukan.
__call__	Tensor	Dunder method yang memanggil forward saja.

Ada banyak jenis layer, dari *dense* untuk FFNN, *konvolusi* untuk CNN, *rekuren* untuk RNN, dan lainnya. Di sini, kami mengimplementasikan yang tercantum pada spesifikasi tugas besar pertama dan kedua, juga yang dapat memudahkan pembuatan arsitektur model CNN dan *Encoder-Decoder* nantinya.

3.3.1. Layer Input

Layer input hanyalah *placeholder* untuk ukuran input model. Ini supaya seluruh model dapat dibangun secara otomatis saat konstruksi. Jika layer ini tidak ada, *building* harus dilakukan secara manual.

Layer Input		
Nama	Tipe Data / Return	Deskripsi
Atribut		
shape	tuple(int)	Ukuran input.
output_shape	tuple(int)	Ukuran output.
built	bool	Flag untuk apakah layer sudah dibangun.
Metode		

forward	Tensor	<i>Forward pass</i> tidak melakukan apa-apa.
build	-	Tidak melakukan apa-apa, hanya toggle flag built.

3.3.2. Layer Dense

Layer dense hanyalah suatu layer MLP atau FFNN standar, yakni suatu kombinasi afin yang diikuti aktivasi. Parameternya hanyalah matriks bobot W dan bias b .

Layer Dense		
Nama	Tipe Data / Return	Deskripsi
Atribut		
input_size	tuple(int)	Ukuran input.
output_size	tuple(int)	Ukuran output.
activation	function	Fungsi aktivasi.
l1_lambda	float	Penalti regularisasi L1.
l2_lambda	float	Penalti regularisasi L1.
rng	numpy.random._generator.Generator	<i>Number generator</i> untuk inisialisasi parameter.
weight_initializer	function	Metode inisialisasi bobot.
bias_initializer	function	Metode inisialisasi bias.
weights	Tensor	Tensor bobot.
bias	Tensor	Tensor bias.
Metode		
forward	Tensor	<i>Forward pass</i> menghitung $f(Wx + b)$.
build	-	Menginisialisasi ukuran input dan parameter weights dan bias.
from_weights	Dense	Konstruktor mudah jika sudah ada bobotnya.

3.3.3. Layer Konvolusi

Layer konvolusi bisa menerapkan *parameter sharing* atau tidak. Yang pertama dinamakan Conv2D sedangkan yang kedua dinamakan LocallyConnected2D. Teknis *forward pass* mereka mengikuti rumus yang ada di pembahasan teori, tetapi implementasinya adalah hibrida dari rumus tersebut dengan teknik *im2col* populer (yaitu cara untuk membuat konvolusi hanya menjadi perkalian matriks). Teknik *im2col* sendiri tidak diimplementasikan karena keterbatasan waktu.

Layer Conv2D		
Nama	Tipe Data / Return	Deskripsi
Atribut		
output_channels	int	Jumlah channel gambar output.
kernel_size	int	Ukuran kernel.
stride	int	Stride saat konvolusi (lompat berapa langkah).
padding	int	Padding sebelum konvolusi (menambahkan piksel di sekeliling).
activation	function	Fungsi aktivasi.
l1_lambda	float	Penalti regularisasi L1.
l2_lambda	float	Penalti regularisasi L1.
rng	numpy.random._generator.Generator	Number generator untuk inialisasi parameter.
weight_initializer	function	Metode inialisasi kernel.
bias_initializer	function	Metode inialisasi bias.
h_in	int	Tinggi gambar input.
w_in	int	Lebar gambar input.
input_channels	int	Jumlah channel input.
h_out	int	Tinggi gambar output.
w_out	int	Lebar gambar output

output_shape	tuple(int)	Ukuran gambar output.
weights	Tensor	Tensor kernel atau filter.
bias	Tensor	Tensor bias.
Metode		
forward	Tensor	<i>Forward pass</i> menghitung $Y_{i,j,k} = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} \sum_{c=0}^{C-1} X_{Si+m, Sj+n, c} K_{m,n,c,k} + b_k.$
build	-	Menginisialisasi ukuran input dan parameter weights dan bias.

Di atas adalah penjelasan untuk kelas Conv2D. Lalu, tabel implementasi untuk LocallyConnected2D sama persis tetapi hanya menggantikan rumus *forward pass*-nya dan *parameter*-nya yang disimpan secara *flattened*.

3.3.4. Layer Pooling

Berikutnya, kami implementasikan layer *pooling* untuk tahap *pooling* dari konvolusi. Selain itu, kami juga membangun layer Flatten (karena juga merupakan operasi penyederhanaan seperti *pooling*) di sini.

pooling.py: Kelas Global			
Nama	Atribut	Metode	Deskripsi
Flatten	output_shape	forward, build	Melakukan <i>flattening</i> dari matriks menjadi array.
GlobalMaxPooling2D	output_shape	forward, build	Melakukan <i>pooling</i> berbasis maksimum seluruh gambar menjadi 1 angka.
GlobalAveragePooling2D	output_shape	forward, build	Melakukan <i>pooling</i> berbasis rata-rata seluruh gambar menjadi 1 angka.
MaxPooling2D	output_shape, pool_size, stride, padding, ...	forward, build	Melakukan <i>pooling</i> berjalan berbasis maksimum.

AveragePooling2D	output_shape, pool_size, stride, padding,...	forward, build	Melakukan <i>pooling</i> berjalan berbasis rata-rata
------------------	---	-------------------	---

Untuk implementasi secara khususnya adalah sebagai berikut. Pertama, layer Flatten hanya melakukan *reshaping* yang memperhitungkan *batch dimension*.

Layer Flatten		
Nama	Tipe Data / Return	Deskripsi
Atribut		
output_shape	tuple(int)	Ukuran output.
Metode		
forward	Tensor	<i>Forward pass</i> hanya <i>reshape</i> input menjadi vektor.
build	-	Hanya menginisialisasi ukuran output.

Berikutnya, tabel berikut adalah implementasi untuk layer *pooling* yang global. Karena yang berbasis maksimum dan rata-rata hanya berbeda di forward pass, di sini kami cantumkan hanya tabel untuk GlobalMaxPooling2D.

Layer GlobalMaxPooling2D		
Nama	Tipe Data / Return	Deskripsi
Atribut		
output_shape	tuple(int)	Ukuran output.
Metode		
forward	Tensor	<i>Forward pass</i> hanya mencari nilai maksimum seluruh gambar: $P = \max_{0 < m, n < Z } Z_{m,n}$
build	-	Hanya menginisialisasi ukuran output.

Terakhir, tabel berikut adalah implementasi untuk layer *pooling* yang lokal (yakni ada filter bergerak). Karena yang berbasis maksimum dan rata-rata hanya berbeda di forward pass, di sini kami cantumkan hanya tabel untuk MaxPooling2D.

Layer MaxPooling2D		
Nama	Tipe Data / Return	Deskripsi
Atribut		
pool_size	int	Ukuran filter maksimum.
stride	int	Stride saat <i>pooling</i> lokal (lompat berapa langkah).
padding	int	Padding sebelum <i>pooling</i> lokal (menambahkan piksel di sekeliling).
output_shape	tuple(int)	Ukuran gambar output.
h_out	int	Tinggi gambar output.
w_out	int	Lebar gambar output.
Metode		
forward	Tensor	<i>Forward pass</i> hanya mencari nilai maksimum setiap <i>patch</i> gambar: $P_{i,j,k} = \max_{0 < m,n < F} Z_{si+m,sj+n,k}$
build	-	Hanya menginisialisasi ukuran output.

3.3.5. Layer Rekuren

Layer rekuren adalah komponen utama RNN. Di sini, kami mengimplementasikan selnya terlebih dahulu (untuk satu time step), lalu menggabungkannya menjadi layer (untuk seluruh sekuens). Pertama, terdapat sel RNN berikut.

Layer SimpleRNNCell		
Nama	Tipe Data / Return	Deskripsi
Atribut		
hidden_size	tuple(int)	Ukuran output.

activation	function	Fungsi aktivasi.
l1_lambda	float	Penalti regularisasi L1.
l2_lambda	float	Penalti regularisasi L1.
rng	numpy.random._generator.Generator	<i>Number generator</i> untuk inisialisasi parameter.
weight_initializer	function	Metode inisialisasi bobot.
bias_initializer	function	Metode inisialisasi bias.
input_size	tuple(int)	Ukuran input.
W_ih	Tensor	Tensor bobot antara input dan hidden state di time step yang sama.
W_hh	Tensor	Tensor bobot antara hidden state dengan hidden state pada time step sebelumnya.
bias	Tensor	Tensor bias.
Metode		
forward	Tensor	<i>Forward pass</i> menghitung $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h).$
build	-	Menginisialisasi ukuran input dan parameter W_ih, W_hh, dan bias.
from_weights	SimpleRNNCell	Konstruktor mudah jika sudah ada bobotnya.

Lalu, sel RNN ini dapat ditumpuk secara horizontal untuk membentuk layer RNN sebagai berikut. Perhatikan adanya opsi output berupa seluruh sekuens maupun hanya time step terakhir.

Layer SimpleRNN		
Nama	Tipe Data / Return	Deskripsi
Atribut		
cell	SimpleRNNCell	Sel RNN yang akan dijalankan pada setiap time step.

return_sequences	bool	<i>Flag</i> untuk meng-output hidden state seluruh sekuens atau time step terakhir saja.
return_state	bool	<i>Flag</i> untuk return hidden state time step terakhir setelah output sebenarnya (biasanya untuk <i>context vector</i>).
output_shape	tuple(int)	Ukuran output.
Metode		
forward	Tensor	<i>Forward pass</i> memanggil cell dengan loop untuk setiap time step.
build	-	Memanggil build untuk cell, dan menginisialisasi ukuran output (sekuens atau tidak).

Hal yang serupa berlaku untuk implementasi LSTM. Cukup pertama mendefinisikan selnya. Di sini, diperlukan *forward pass* yang lebih canggih menggunakan gate dan *cell state*.

Layer LSTMCell		
Nama	Tipe Data / Return	Deskripsi
Atribut		
hidden_size	tuple(int)	Ukuran output.
activation	function	Fungsi aktivasi.
l1_lambda	float	Penalti regularisasi L1.
l2_lambda	float	Penalti regularisasi L1.
rng	numpy.random._generator.Generator	<i>Number generator</i> untuk inisialisasi parameter.
weight_initializer	function	Metode inisialisasi bobot.
bias_initializer	function	Metode inisialisasi bias.
input_size	tuple(int)	Ukuran input.
W_ih	Tensor	Tensor bobot antara input dan hidden state di time step yang sama untuk

		keempat gate (dikonkatenasi menjadi satu matriks besar).
W_hh	Tensor	Tensor bobot antara hidden state dengan hidden state pada time step sebelumnya untuk keempat gate (dikonkatenasi menjadi satu matriks besar).
bias	Tensor	Tensor bias untuk keempat gate (dikonkatenasi menjadi satu matriks besar).
Metode		
forward	Tensor	<i>Forward pass</i> menghitung: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$ $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$ $\hat{C}_t = \sigma(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$ $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$ $c_t = f_t \odot c_{t-1} + i_t \odot \hat{C}_t$ $h_t = o_{t-1} \odot \tanh(c_t)$
build	-	Menginisialisasi ukuran input dan parameter W_ih, W_hh, dan bias.
from_weights	LSTMCell	Konstruktor mudah jika sudah ada bobotnya.

Lalu, sel LSTM lagi ditumpuk secara horizontal untuk membentuk layer LSTM.

Layer LSTM		
Nama	Tipe Data / Return	Deskripsi
Atribut		
cell	LSTMCell	Sel LSTM yang akan dijalankan pada setiap time step.
return_sequences	bool	<i>Flag</i> untuk meng-output hidden state seluruh sekuens atau time step terakhir saja.

return_state	bool	<i>Flag</i> untuk return <i>hidden state</i> dan <i>cell state</i> time step terakhir setelah output sebenarnya (biasanya untuk <i>context vector</i>).
output_shape	tuple(int)	Ukuran output.
Metode		
forward	Tensor	<i>Forward pass</i> memanggil <i>cell</i> dengan loop untuk setiap time step.
build	-	Memanggil <i>build</i> untuk <i>cell</i> , dan menginisialisasi ukuran output (sekuens atau tidak).

3.3.6. Layer Embedding

Nanti, kami akan mengembangkan fungsi utilitas untuk mentokenisasi teks menjadi sekuens id. Tetapi, secara praktis, lebih efektif jika teks sekuens tersebut (yang mungkin sangat berdimensi sangat tinggi untuk *vocabulary* yang besar) direpresentasikan dalam suatu vektor yang lebih padat (berdimensi lebih rendah) tetapi tetap menyimpan informasi semantik tentang teksnya. Ini adalah fungsi dari layer *embedding*, yang akan melakukan konversi sekuens id menjadi representasi vektor tersebut. Tentu, representasi ini *learnable* (bisa dipelajari dalam training). Berikut implementasinya.

Layer Embedding		
Nama	Tipe Data / Return	Deskripsi
Atribut		
vocab_size	int	Banyak kata dalam <i>vocabulary</i> .
embedding_dim	int	Dimensi ruang representasi vektor <i>embedding</i> .
l1_lambda	float	Penalti regularisasi L1.
l2_lambda	float	Penalti regularisasi L1.
rng	numpy.random._generator.Generator	<i>Number generator</i> untuk inisialisasi parameter.
weight_initializer	function	Metode inisialisasi bobot.

weights	Tensor	Matriks yang mengkonversikan sekuens menjadi vektor <i>embedding</i> , seperti suatu “lookup matrix”.
output_shape	list(int)	Ukuran tensor vektor <i>embedding</i> .
Metode		
forward	Tensor	<i>Forward pass</i> cukup mengembalikan kolom dari weights yang bersesuaian.
build	-	Menginisialisasi ukuran output.
from_weights	Embedding	Konstruktor mudah jika sudah ada bobotnya.

3.3.7. Layer RMSNorm

Sebagai artefak dari tugas besar pertama, kami melakukan *porting* layer RMSNorm menjadi suatu kelas Layer yang diimplementasikan di sini. Ingat bahwa RMSNorm melakukan normalisasi pada nilai aktivasi (sudah jarang digunakan pada arsitektur *neural network* sekarang, tetapi dapat membantu untuk stabilitas).

Layer RMSNorm		
Nama	Tipe Data / Return	Deskripsi
Atribut		
eps	float	Konstan aditif untuk <i>numerical stability</i> (agar tidak membagi dengan nol)
gamma	Tensor	Pengali skala untuk statistik RMSNorm.
output_shape	list(int)	Ukuran output.
Metode		
forward	Tensor	<i>Forward pass</i> mengikuti rumus: $\bar{a}_i = \frac{a_i}{\sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}} \gamma_i.$
build	-	Menginisialisasi ukuran output dan tensor gamma.

3.4. Kelas Model

Akhirnya, semua komponen *neural network* dan jenis-jenis layer penting yang telah diimplementasikan dapat disatukan menjadi satu *container* model. Wadah ini akan menampung semua layer-layer yang ada dan komposisi mereka (karena ingat *neural network* hanyalah komposisi fungsi-fungsi). Dengan ini, *forward* dan *backward pass* seluruh model dapat dilakukan sekaligus. Dia juga yang akan memanggil Optimizer untuk melakukan training. Implementasinya adalah dalam kelas Model sebagai berikut.

Kelas Model		
Nama	Tipe Data / Return	Deskripsi
Atribut		
seed	int	<i>Seed</i> untuk <i>number generator</i> .
rng	numpy.random._generator.Generator	<i>Number generator</i> untuk permutasi batch saat training.
layers	list[Layer]	List semua layer yang ada pada model.
optimizer	Optimizer	Metode optimisasi yang digunakan.
loss_fn	function	Fungsi loss yang digunakan.
Metode		
forward	Tensor	Melakukan <i>forward pass</i> dengan memanggil seluruh layer pada <i>layers</i> secara berurutan (komposisi fungsi).
add	-	Menambahkan layer baru ke model.
build	-	Menginisialisasikan ukuran input. Lalu, lakukan <i>build</i> pada semua layer dengan ukuran input tiap layer adalah ukuran output layer sebelumnya.
compile	-	Menginisialisasikan metode optimisasi dan fungsi loss yang digunakan.
predict	np.ndarray	Jika model sudah dibangun, melakukan prediksi berdasarkan input data.
_regularization_loss	Tensor	Menghitung total kontribusi regularisasi terhadap loss dengan

		memanggil <code>regularization_loss</code> pada tiap layer.
<code>evaluate</code>	<code>np.ndarray</code>	Jika model sudah dibangun, melakukan menghitung loss prediksi model berdasarkan input data dan target.
<code>fit</code>	<code>dict</code>	Melakukan <i>training</i> atau pelatihan semua parameter tiap layer pada model. Mendukung <i>batch inference</i> (karena semua layer telah dirancang agar dimensi <i>axis</i> pertama adalah untuk <i>batch</i>). Menggunakan <code>tqdm</code> untuk <i>verbose</i> yang bersih. Mendukung split validasi juga.
<code>summary</code>	-	Memprint tabel untuk arsitektur model, yang meliputi: jenis setiap layer, dimensi output shape setiap layer, dan jumlah parameter (<i>trainable</i> dan tidak) dari tiap layer.
<code>save</code>	-	Menyimpan nilai bobot parameter setiap layer dalam file <code>npz</code> .
<code>load</code>	-	Memuat nilai-nilai bobot dari file <code>npz</code> (yang sudah sesuai dengan arsitektur model) pada parameter.

Di sini, perhatikan bahwa kelas `Model` secara dirancang untuk bersifat lumayan *high-level* tetapi sangat *customizable* seperti Keras. Fitur seperti melakukan prediksi dan evaluasi, pelatihan, `summary`, pemuatan dan penyimpanan bobot semua didukung.

3.5. Fungsi Utilitas

Di bagian terakhir ini, kami mengembangkan tiga fungsi utilitas yang menjembatani arsitektur yang akan kami bikin dengan package yang sudah dibuat. Khususnya, diperlukan cara untuk *preprocessing* data gambar dan caption menjadi fitur yang dapat dicerna oleh suatu *neural network*. Lalu, diperlukan juga fungsi untuk menerjemahkan bobot pada model yang dilatih dari Keras ke bobot untuk kelas `Model` kustom package kami.

3.5.1. Utilitas Gambar

Pertama, dibuat fungsi utilitas untuk pemrosesan gambar. Tujuannya adalah untuk mengkonversikan file gambar menjadi suatu array 2D numerik yang bisa menjadi input suatu *neural network*.

image_utils.py: Fungsi Global			
Nama	Parameter	Tipe Return	Deskripsi
load_image	string, int	np.ndarray	Memproses file gambar dengan mengkonversi menjadi suatu array numerik (dengan ukuran tertentu) yang ternormalisasi.
load_batch	list[string], int	list[np.ndarray]	Memproses banyak file gambar sekaligus.

3.5.2. Utilitas Teks

Berikutnya, diperlukan fungsi utilitas untuk pemrosesan teks. Ini akan digunakan pada data caption. Konstruksinya adalah dengan membuat kelas kustom bernama TextTokenizer yang membangun *vocabulary* yang berlaku pada dataset, sehingga bisa dengan mudah mentranslasikan teks (token kata) menjadi sekuens numerik dan juga sebaliknya. Kelas ini juga mendukung penyimpanan *vocabulary* tersebut ke dalam disk. Dengan pemrosesan ini, teks menjadi larik numerik yang bisa menjadi input untuk *neural network*.

Kelas TextTokenizer		
Nama	Tipe Data / Return	Deskripsi
Atribut		
token_to_id	dict	Dictionary untuk kata ke angka.
id_to_token	dict	Dictionary untuk angka ke kata.
vocab_size	int	Banyak kata di <i>vocabulary</i> .
Metode		
clean_text	string	Membuat seluruh teks berhuruf kecil dan membuang karakter <i>non-alphanumeric</i> .
build_vocab	-	Membangun <i>vocabulary</i> di token_to_id dan

		id_to_token dari sebuah teks masukan.
pad_sequence	list[list[string]]	Memberikan <i>padding</i> dengan id 0 untuk semua sekuens masukan agar sepanjang yang paling panjang (atau masukannya).
encode	string	Menerjemahkan sekuens angka ke kalimat.
decode	list[string]	Menerjemahkan kalimat ke sekuens angka.
save_vocab	-	Simpan <i>vocabulary</i> dalam file json.
load_vocab	-	Memuat <i>vocabulary</i> dari file json.

3.5.3. Keras Weight Loader

Terakhir, walaupun package kami sudah mendukung *backpropagation* sepenuhnya, karena model yang dikembangkan lumayan kompleks dan berat (terutama yang *encoder-decoder*), pelatihan akan tetap dilakukan dalam Keras. Tetapi, untuk inferensi (*forward pass*) pada test set atau data baru akan tetapi menggunakan package kami. Oleh karena itu, perlu cara untuk memuat bobot-bobot parameter model dari instansi Keras ke instansi model di package kami. Untungnya, package kami sudah dirancang secara khusus untuk mengurutkan urutan dan *axes* parameter model pada tiap layer sesuai dengan yang di Keras.

model_loader.py: Fungsi Global			
Nama	Parameter	Tipe Return	Deskripsi
load_keras_weights	keras.Model, nn.Model, bool	-	Memindahkan bobot-bobot dari model keras ke model kustom.

BAB IV

HASIL PENGUJIAN DAN ANALISIS

4.1. CNN: Image Classification

Pada pengujian ini, evaluasi model Convolutional Neural Network (CNN) dilakukan untuk menyelesaikan tugas klasifikasi citra multi-kelas. Dataset yang digunakan adalah Intel Image Classification yang terdiri dari sekitar 25.000 citra pemandangan alam dan lanskap perkotaan. Citra-citra tersebut diklasifikasikan ke dalam enam kelas utama, yaitu *buildings*, *forest*, *glacier*, *mountain*, *sea*, dan *street*. Penggunaan dataset dengan variasi kelas yang beragam ini bertujuan untuk menguji kemampuan arsitektur CNN dalam mengekstraksi dan membedakan fitur visual hierarkis yang kompleks dari berbagai kondisi lingkungan. Seluruh dataset telah dipartisi ke dalam tiga bagian, yaitu *set training*, *set validation*, dan *set test*.

Pengujian difokuskan pada pengamatan dua metrik utama, yaitu akurasi dan Macro F1-Score. Penggunaan Macro F1-Score sangat krusial dalam evaluasi ini guna memastikan bahwa kemampuan prediksi model dievaluasi secara adil pada setiap kelas secara individu. Hal ini memberikan representasi performa yang lebih akurat dan seimbang dibandingkan hanya menggunakan metrik akurasi secara keseluruhan.

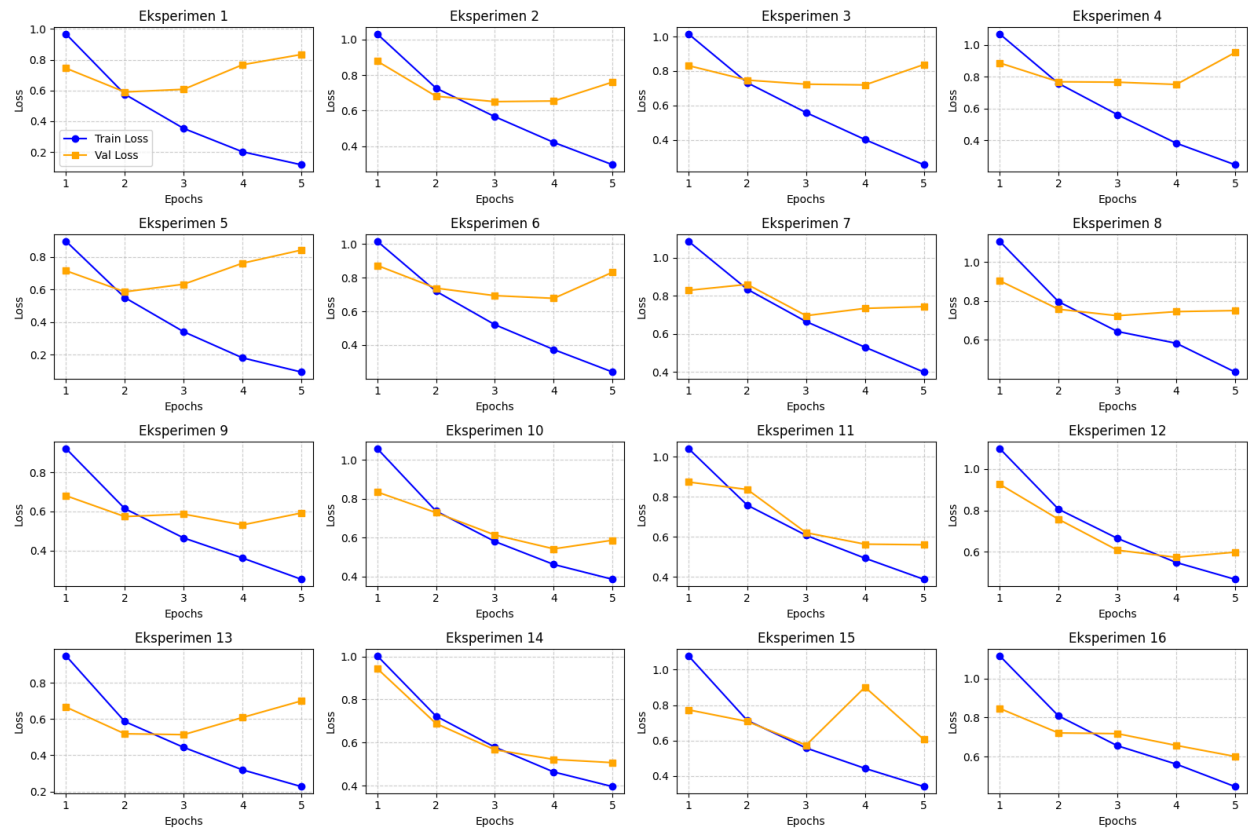
Untuk mendapatkan arsitektur yang paling optimal, tahap pertama pengujian terdiri dari 16 eksperimen hyperparameter menggunakan *framework* dari Keras. Variasi *hyperparameter* yang diuji di antaranya kombinasi jumlah *layer* konvolusi (2 dan 3 *layer*), jumlah *filter* (32, 64, dan 128 *filter* serta 64, 128, dan 256 *filter*), ukuran *filter/kernel* (3x3 dan 5x5), serta tipe operasi *pooling* (*max pooling* dan *average pooling*). Selanjutnya, pengujian juga dilakukan menggunakan *library from scratch*. Model diuji untuk membandingkan secara langsung pengaruh penerapan *parameter sharing* dengan arsitektur tanpa *parameter sharing* terhadap efisiensi, ledakan jumlah parameter, dan performa generalisasi model. Eksperimen yang dilakukan secara rinci ditunjukkan pada tabel di bawah ini.

Eksperimen	Layer	Filter	Kernel size	Pooling
1	2	[32, 64]	(3, 3)	max
2	2	[32, 64]	(3, 3)	avg
3	2	[32, 64]	(5, 5)	max
4	2	[32, 64]	(5, 5)	avg
5	2	[64, 128]	(3, 3)	max
6	2	[64, 128]	(3, 3)	avg

7	2	[64, 128]	(5, 5)	max
8	2	[64, 128]	(5, 5)	avg
9	3	[32, 64, 128]	(3, 3)	max
10	3	[32, 64, 128]	(3, 3)	avg
11	3	[32, 64, 128]	(5, 5)	max
12	3	[32, 64, 128]	(5, 5)	avg
13	3	[64, 128, 256]	(3, 3)	max
14	3	[64, 128, 256]	(3, 3)	avg
15	3	[64, 128, 256]	(5, 5)	max
16	3	[64, 128, 256]	(5, 5)	avg

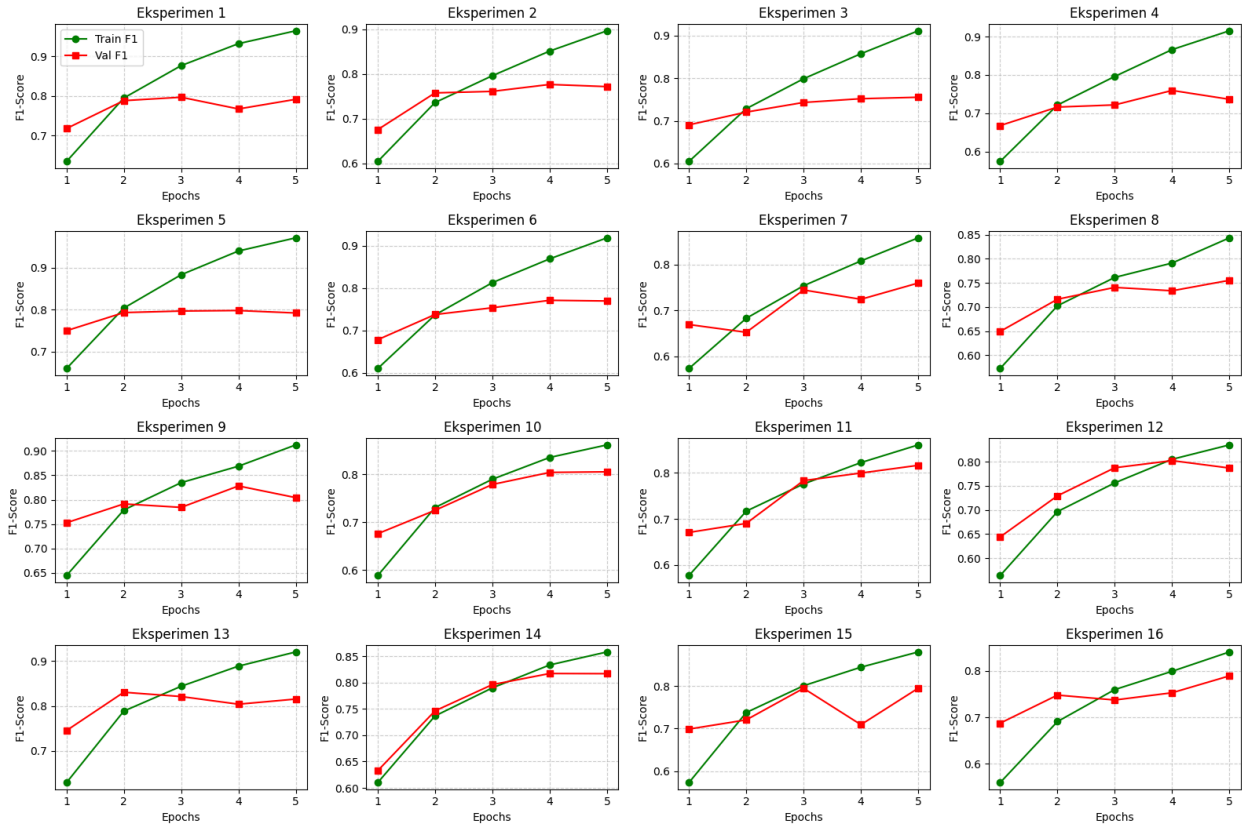
Hasil *training* dan *validation loss* ditunjukkan pada grafik di bawah ini.

Grafik Training dan Validation Loss



Hasil *training* dan *validation F1-Score* ditunjukkan pada grafik di bawah ini.

Grafik Training and Validation F1-Score



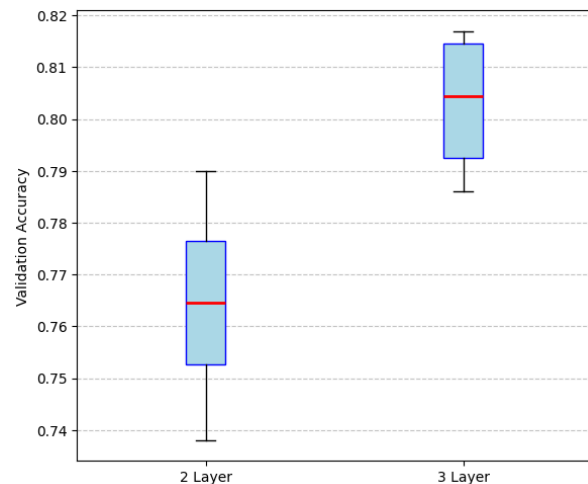
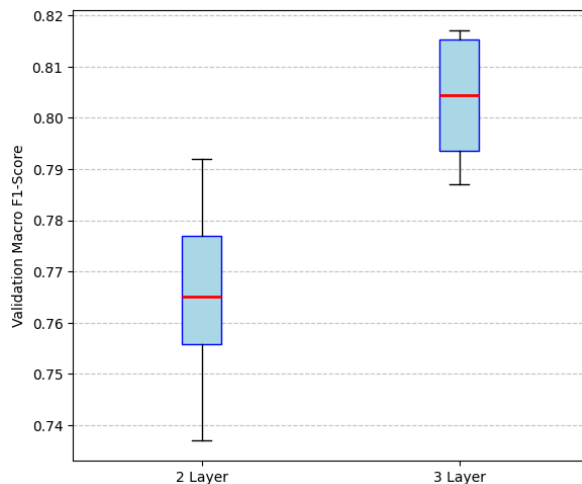
Adapun hasil F1-Score dan akurasi pada epoch terakhir pada masing-masing eksperimen ditunjukkan pada tabel di bawah ini.

Eksperimen	Layer	Filter	Kernel size	Pooling	val_macro_f1	val_accuracy
1	2	[32, 64]	(3, 3)	max	0.792	0.790
2	2	[32, 64]	(3, 3)	avg	0.772	0.770
3	2	[32, 64]	(5, 5)	max	0.755	0.752
4	2	[32, 64]	(5, 5)	avg	0.737	0.738
5	2	[64, 128]	(3, 3)	max	0.792	0.790
6	2	[64, 128]	(3, 3)	avg	0.770	0.772
7	2	[64, 128]	(5, 5)	max	0.760	0.759
8	2	[64, 128]	(5, 5)	avg	0.756	0.753
9	3	[32, 64, 128]	(3, 3)	max	0.804	0.804
10	3	[32, 64, 128]	(3, 3)	avg	0.805	0.805

11	3	[32, 64, 128]	(5, 5)	max	0.816	0.814
12	3	[32, 64, 128]	(5, 5)	avg	0.787	0.786
13	3	[64, 128, 256]	(3, 3)	max	0.815	0.817
14	3	[64, 128, 256]	(3, 3)	avg	0.817	0.816
15	3	[64, 128, 256]	(5, 5)	max	0.795	0.794
16	3	[64, 128, 256]	(5, 5)	avg	0.789	0.788

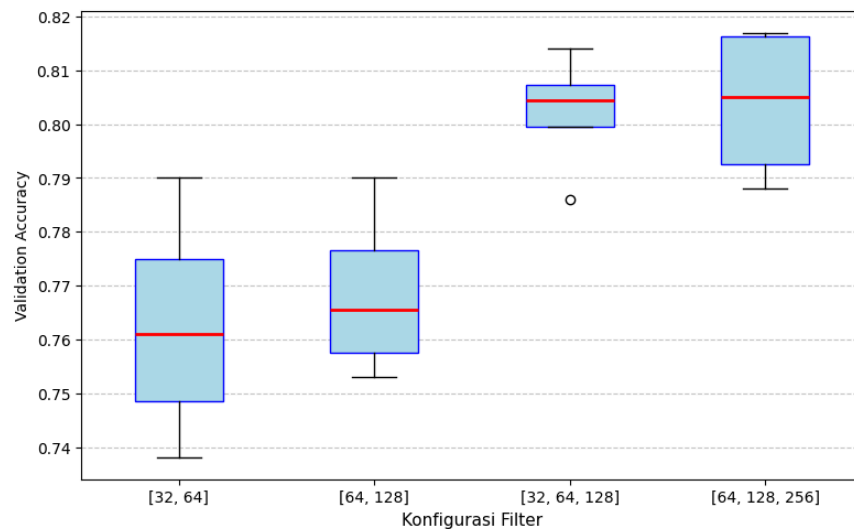
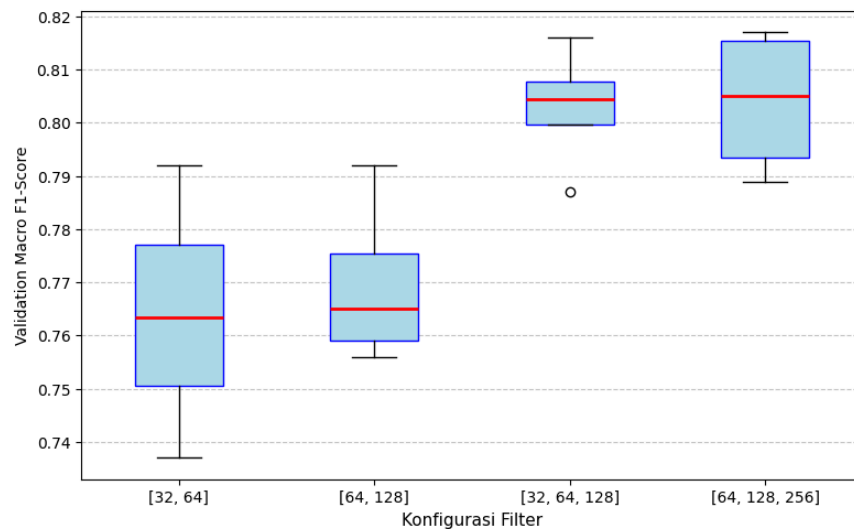
4.1.1. Pengaruh Jumlah Layer Konvolusi

Pada pengujian ini, jumlah *layer* konvolusi yang digunakan terbatas hanya pada 2 *layer* konvolusi dan 3 *layer* konvolusi. Model CNN dengan 3 *layer* konvolusi secara konsisten mendominasi pada kedua metrik evaluasi yang diuji, yakni Macro F1-Score dan Accuracy seperti yang ditunjukkan pada kedua grafik di bawah ini. Selain itu, seperti yang ditunjukkan pada tabel sebelumnya, nilai *Macro F1-Score* tertinggi pada model CNN dengan 2 *layer* konvolusi hanya mencapai 0.792, yaitu pada eksperimen 1 dan 5. Sementara itu, model CNN dengan 3 *layer* konvolusi mampu meningkatkan performa pada kedua metrik evaluasi tersebut sampai melebihi angka 0.80. Metrik evaluasi tertinggi yaitu *Macro F1-Score* sebesar 0.817 pada eksperimen 14 dan *Accuracy* sebesar 0.817 pada eksperimen 13. Dengan demikian, penambahan jumlah *layer* konvolusi memberikan peningkatan performa yang signifikan pada model CNN. Akan tetapi, karena percobaan yang terbatas, akan terdapat suatu *threshold* ketika penambahan *layer* konvolusi tidak memberikan peningkatan performa model atau justru rentan terhadap *overfitting*.



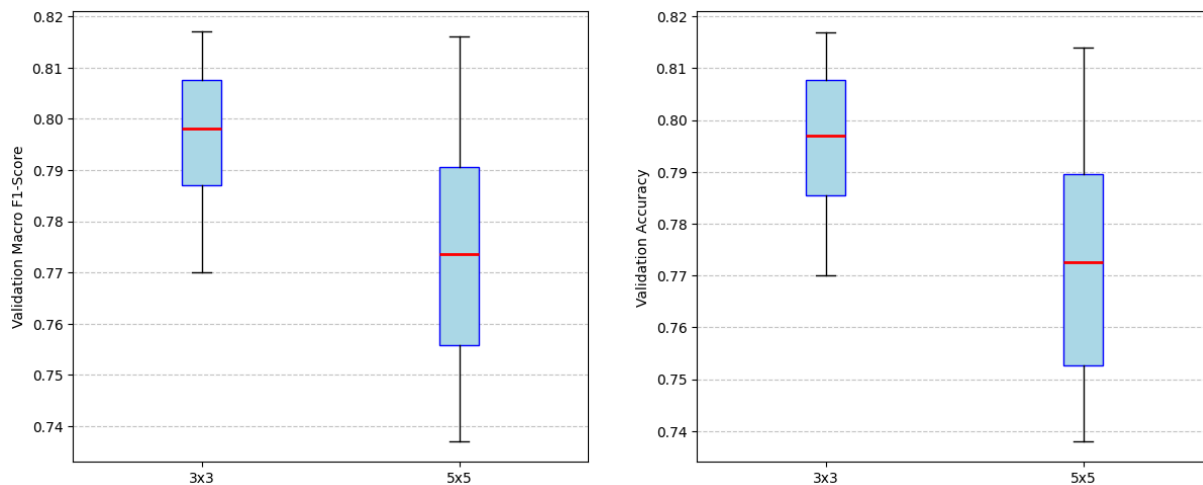
4.1.2. Pengaruh Banyak *Filter*

Pada pengujian ini, banyak *filter* yang digunakan terbatas hanya pada [32, 64], [64, 128], [32, 64, 128], dan [64, 128, 256]. Penambahan banyak *filter* secara langsung memperbesar kapasitas representasi model untuk mengekstrak fitur. Seperti yang ditunjukkan pada tabel sebelumnya, apabila kita perhatikan eksperimen 10 (*filter* [32, 64, 128]) dengan eksperimen 14 (*filter* [64, 128, 256]), nilai *Macro F1-Score* meningkat dari 0.805 menjadi 0.817 serta nilai *Accuracy* meningkat dari 0.805 menjadi 0.816 pada penggunaan *filter* lebih banyak. Kapasitas *filter* yang lebih banyak seperti [64, 128, 256] memungkinkan model menangkap lebih banyak variasi fitur dalam satu *layer*. Meskipun menyebabkan nilai *train loss* turun lebih curam, jika dikombinasikan dengan jenis pooling dan ukuran kernel yang tepat, penambahan filter terbukti efektif meningkatkan kemampuan generalisasi model terhadap data yang belum pernah dilihat seperti yang ditunjukkan pada kedua grafik di bawah ini.



4.1.3. Pengaruh Ukuran *Filter*

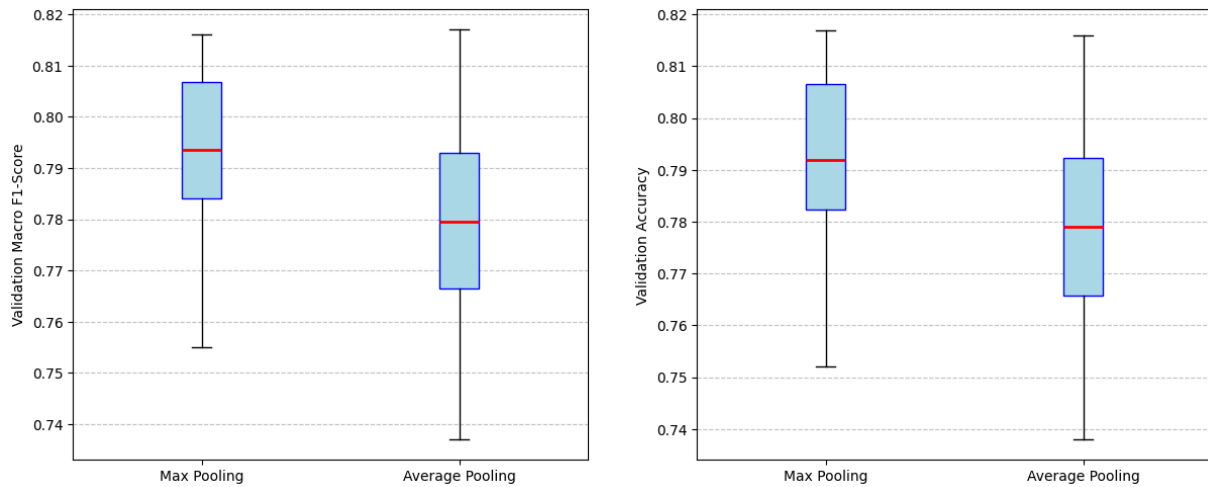
Pada pengujian ini, ukuran *filter* yang digunakan terbatas hanya pada ukuran 3x3 dan 5x5. Pada model CNN dengan 2 *layer* konvolusi, penggunaan kernel dengan ukuran 3x3 seperti pada eksperimen 1, nilai *macro F1-Score* sebesar 0.792 lebih unggul dibandingkan kernel dengan ukuran 5x5 seperti pada eksperimen 3 yang nilai *macro F1-Score* bernilai 0.755. Selain itu, nilai *Accuracy* pada eksperimen 1 dan 3 secara berturut-turut adalah 0.790 dan 0.752. *Filter* ukuran 3x3 mampu menangkap detail lokal yang lebih halus dan presisi dengan jumlah parameter yang lebih sedikit. Meskipun *filter* berukuran 5x5 dapat memperoleh nilai tinggi seperti pada eksperimen 11 dengan *macro F1-Score* bernilai 0.816 dan *Accuracy* bernilai 0.814, tetapi secara keseluruhan performa *filter* ukuran 3x3 jauh lebih konsisten dalam menjaga metrik *macro F1-Score* dan *Accuracy* lebih baik seperti yang ditunjukkan pada kedua grafik di bawah ini. Dengan demikian, penggunaan *filter* berukuran lebih kecil, yaitu ukuran 3x3, terbukti jauh lebih efektif dan stabil dibandingkan *filter* ukuran 5x5.



4.1.4. Pengaruh Jenis *Pooling Layer*

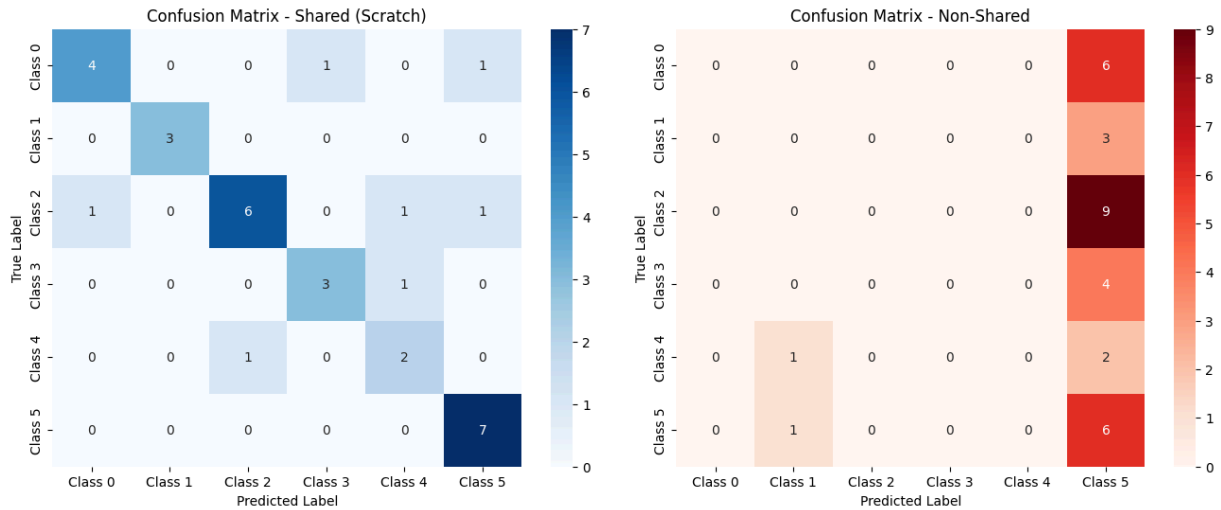
Pada pengujian ini, jenis *pooling layer* yang digunakan adalah *Max Pooling* dan *Average Pooling*. Hasil eksperimen menunjukkan bahwa penggunaan *Max Pooling* secara umum memberikan performa yang lebih unggul dan stabil dibandingkan *Average Pooling*. Sebagai contoh, pada model CNN dengan 2 *layer* konvolusi, penggunaan *Max Pooling* pada eksperimen 1 berhasil mencapai nilai *Macro F1-Score* sebesar 0.792 dan *Accuracy* sebesar 0.790. Nilai ini lebih tinggi jika dibandingkan dengan eksperimen 2 yang menggunakan *Average Pooling* pada konfigurasi parameter yang identik, dengan perolehan *Macro F1-Score* hanya mencapai 0.772 dan *Accuracy* 0.770. Meskipun pada arsitektur yang lebih dalam, seperti 3 *layer* konvolusi, *Average Pooling* mampu memberikan hasil yang lebih baik, ditunjukkan pada eksperimen 14 yang mendapat *Macro F1-Score* sebesar 0.817, tetapi *Max Pooling* tetap mendominasi di sebagian besar skenario pengujian. Hal ini divalidasi oleh visualisasi grafik *boxplot*, dengan distribusi performa model dengan *Max Pooling* memiliki nilai median yang lebih tinggi dan

sebaran performa yang cenderung lebih baik secara keseluruhan. Jadi, dapat disimpulkan bahwa *Max Pooling* merupakan pendekatan yang lebih andal pada arsitektur model CNN.



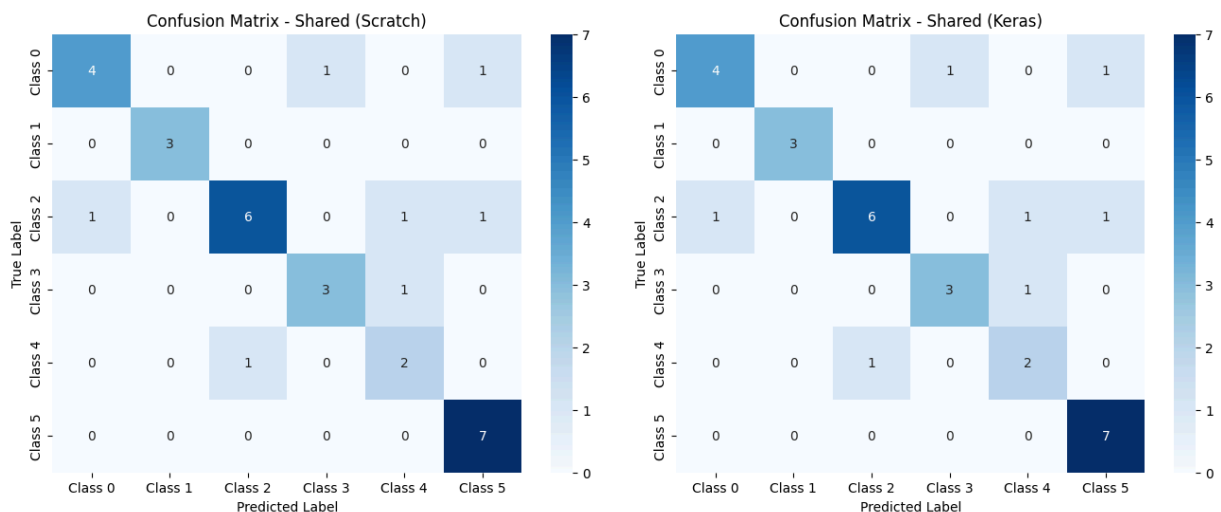
4.1.5. Pengaruh *Parameter Sharing*

Parameter sharing merupakan fondasi utama dari CNN yang memberikan model sifat mampu mengenali objek terlepas dari posisinya di dalam gambar. Pada evaluasi ini, model CNN dengan parameter sharing memiliki kinerja efisien karena satu set *filter* digunakan berulang melintasi seluruh area piksel. Sebaliknya, saat model diubah menggunakan *non-shared parameter*, model dituntut untuk mempelajari bobot yang berbeda untuk setiap titik area piksel. Hal ini menyebabkan fenomena ledakan jumlah parameter yang memicu *out-of-memory* dan *overfitting*. Untuk tetap mendemonstrasikan evaluasi metrik, perbandingan dilakukan menggunakan model skala-kecil. Dari model skala-kecil ini pun terbukti bahwa arsitektur *non-shared parameter* (kanan) cenderung mengalami *overfitting* yang ekstrem dan tidak mampu menggeneralisasi pola sebaik pada model *shared-parameter* (kiri) seperti yang ditunjukkan oleh kedua gambar *confusion matrix* di bawah ini.



4.1.6. Perbandingan Keras vs Scratch

Pada tahap evaluasi ini, dilakukan perbandingan kinerja untuk model *parameter sharing* antara *library* Keras dengan *library from scratch*. Evaluasi ini menggunakan arsitektur terbaik dari pengujian sebelumnya, yaitu eksperimen 14 dengan 3 *layer* konvolusi, *filter* sebanyak 64, 128, dan 256 buah, ukuran *filter* 3x3, serta *pooling* menggunakan *average*. Model terbaik ini memiliki total parameter sebanyak 4.565.574 parameter. Sebagai catatan teknis, proses komputasi inferensi *library from scratch* menggunakan alokasi memori yang jauh lebih masif. Oleh karena itu, untuk menghindari *Memory Error*, tanpa mengurangi keabsahan pembuktian, pengujian inferensi ini dilakukan menggunakan subset acak yang representatif dari data uji.

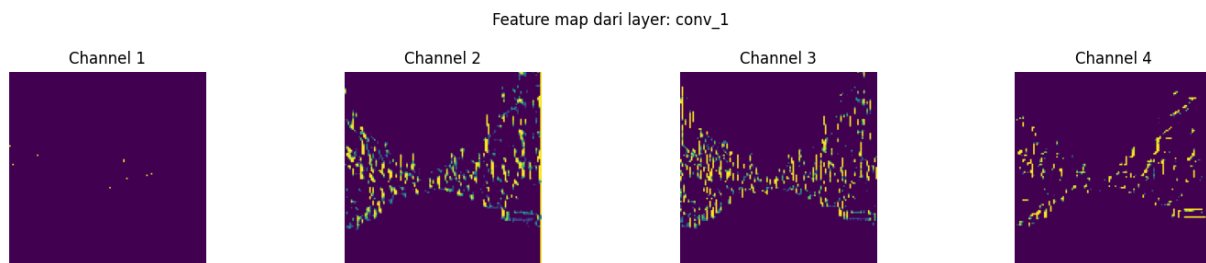


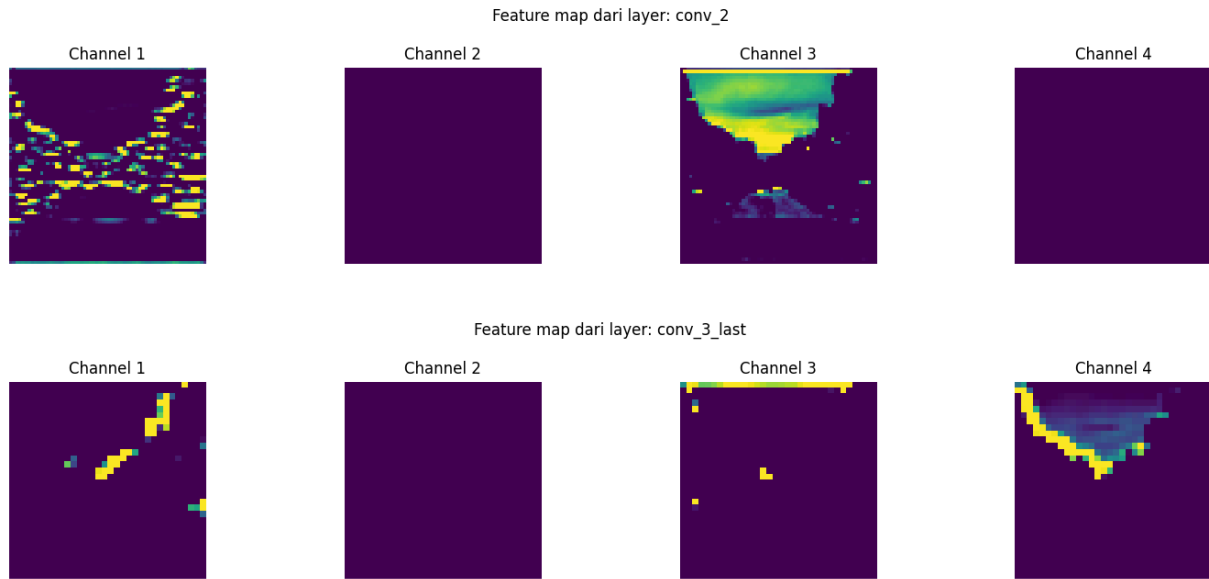
Berdasarkan hasil pengujian, performa antara kedua model menghasilkan nilai yang 100% identik, baik model Keras maupun model *library from scratch* sama-sama menghasilkan nilai Macro F1-Score sebesar 0.7790 dengan nilai *confusion matrix* yang identik tiap entrinya. Kesamaan pada metrik performa dan hasil akhir prediksi ini memberikan kesimpulan bahwa *library from scratch* telah diimplementasikan dengan sempurna dan akurat seperti *library from Keras*. Salah satu perbedaan di antara keduanya terletak pada efisiensi waktu komputasi, yang mana Keras dieksekusi secara instan akibat optimisasi *backend* C++, sementara implementasi *from scratch* berjalan lebih lambat di atas *native* Python, tetapi tetap berfungsi baik sebagai instrumen validasi arsitektur CNN.

4.1.7. Visualisasi Grad-CAM

Untuk memahami proses pengambilan keputusan oleh model CNN, pengujian ini mengimplementasikan teknik *Gradient-weighted Class Activation Mapping* (Grad-CAM) merujuk pada metodologi yang diusulkan oleh Selvaraju et al. (2019). Berbeda dengan metrik evaluasi kuantitatif seperti akurasi atau macro F1-Score, visualisasi ini bertujuan untuk memberikan interpretabilitas terhadap arsitektur jaringan. Hal ini penting untuk membuktikan bahwa model tidak sekadar menghafal noise pada dataset, melainkan benar-benar mengidentifikasi representasi fitur yang relevan dengan kelas target. Visualisasi diterapkan pada model terbaik hasil *training* Keras, yaitu eksperimen 14 dengan 3 *layer* konvolusi, *filter* 64, 128, dan 256 buah, ukuran *filter* 3x3, serta *pooling* menggunakan *average*.

Pada tahap pertama, visualisasi *intermediate feature maps* dilakukan untuk mengamati transformasi data gambar di dalam jaringan. Pada layer konvolusi awal (*conv_1*), filter cenderung menangkap fitur visual tingkat rendah seperti deteksi tepi, garis kontur, dan warna. Semakin dalam citra diproses hingga mencapai layer terakhir (*conv_3_last*), representasi *feature map* mengalami reduksi resolusi akibat operasi *pooling* dan berubah menjadi representasi abstrak yang kaya akan informasi semantik kelas, meskipun semakin sulit dikenali secara visual oleh mata manusia. Berikut ini adalah visualisasi *intermediate feature maps* untuk 4 channel pertama.





Hasil pengujian menunjukkan bahwa model mampu melokalisasi objek utama pendefinisi kelas dengan sangat presisi. Model menunjukkan kemampuan lokalisasi objek. Hal ini mengonfirmasi keandalan arsitektur dan efektivitas parameter sharing dalam mengekstraksi fitur penentu keputusan secara independen. Berikut ini adalah analisis keputusan model yang disandingkan antara gambar asli dengan *heatmap* Grad-Cam.

Analisis Keputusan Model (Prediksi: street)

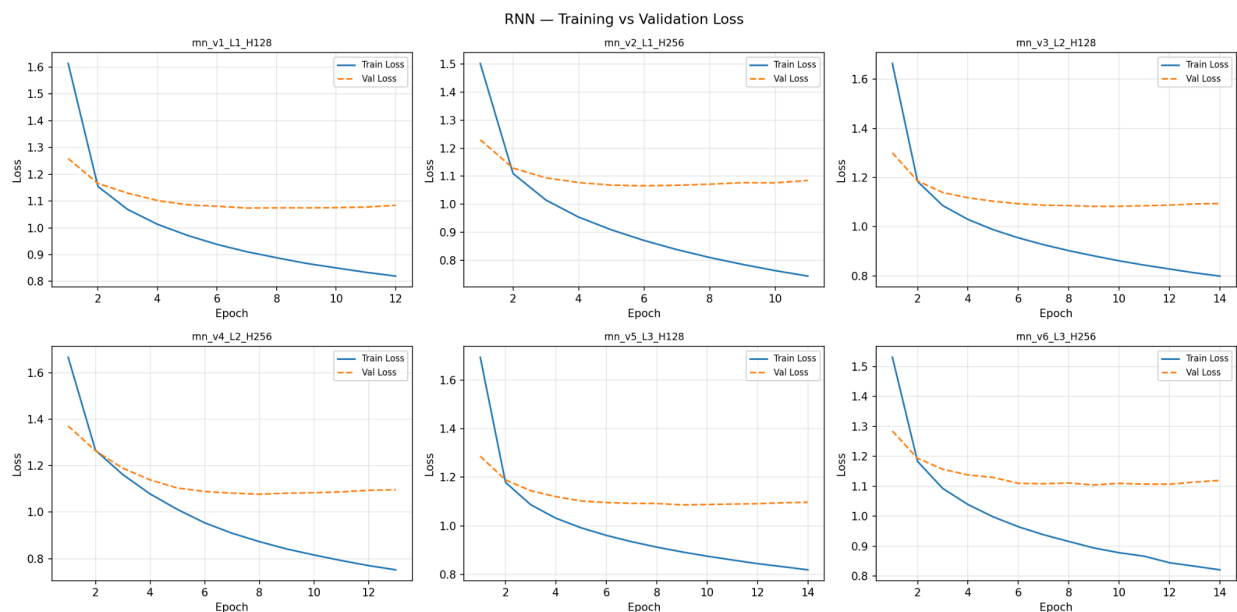


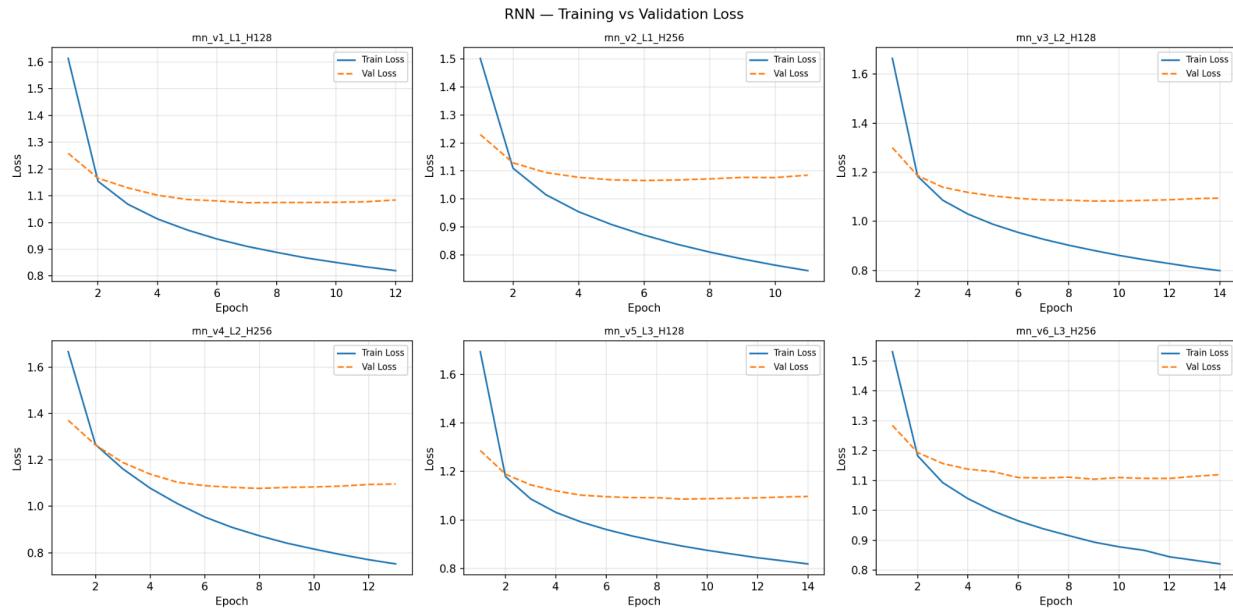
4.2. RNN dan LSTM: Image Captioning

Pada pengujian ini, dibangun model *Encoder-Decoder* berdasarkan arsitektur “Show and Tell” yang telah dijelaskan di pembahasan. Khususnya, kita fokuskan pada variasi *decoder* dengan RNN dan LSTM. Seperti yang telah dijelaskan, model ini bertujuan untuk menyelesaikan permasalahan *image captioning*. Untuk menganalisis performa model *encoder-decoder* ini, kami akan melakukan eksperimentasi performa model dengan 6 variansi untuk kedua RNN dan LSTM (sehingga total ada 12 model) dengan spesifikasi sebagai berikut.

Variasi	Jumlah Layer	Banyak Hidden State
1	1	128
2	1	256
3	2	128
4	2	256
5	3	128
6	3	258

Dataset dibagi menjadi *train set*, *validation set*, dan *test set*. Lalu, pelatihan keenam variasi model dilakukan di Keras berdasarkan *train set*. Hasil *training curve* dengan *training loss* (kurva biru) dan *validation loss* (kurva jingga) yang diperoleh terlihat di figur berikut.

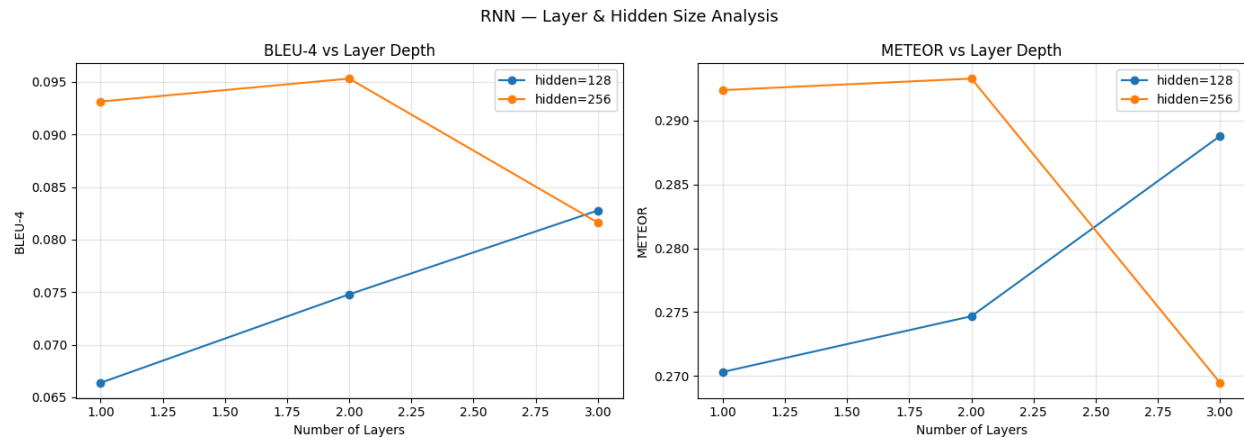




Terlihat pola yang serupa pada semua variasi untuk kedua model RNN dan LSTM. Hampir semua jenis variasi mengalami konvergensi pada area sekitar epoch ke-3 (terlihat dari *validation loss*), sehingga proses pelatihan dapat berakhir cepat dan beberapa bahkan terhenti karena *early stop* sebelum mencapai iterasi maksimal. Jarak antara *training* dan *validation* loss setelah banyak epoch juga menunjukkan arsitektur yang dikembangkan masih bisa terus diperbaiki. Terakhir, karena hasil *training curve* sangat homogen antara model, interpretasinya tidak akan digunakan untuk analisis dan perbandingan berikutnya.

4.2.1. Perbandingan Jumlah Layer: RNN

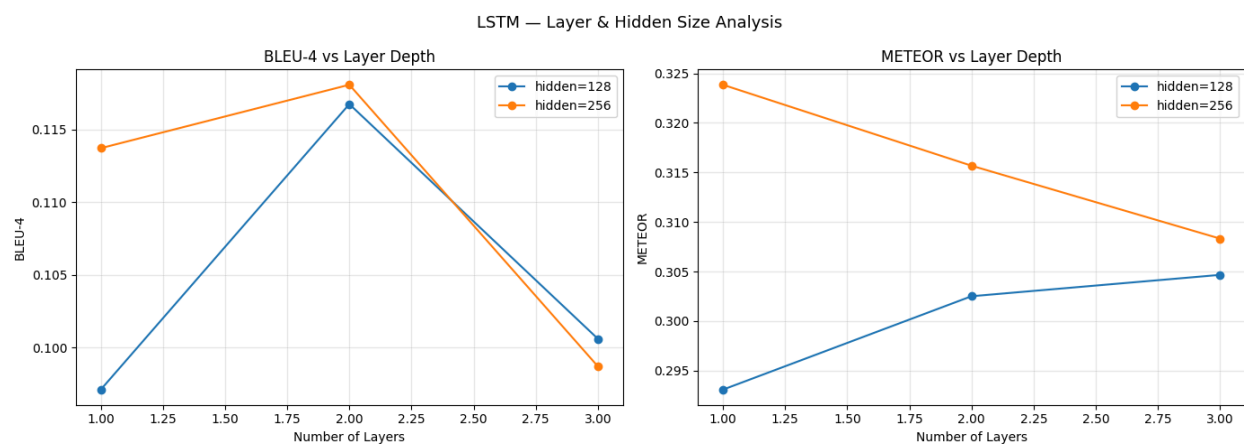
Grafik garis di bawah menunjukkan pengaruh jumlah layer terhadap performa model RNN dengan metrik BLEU-4 maupun METEOR. Secara mutlak global, terlihat bahwa untuk RNN, nilai BLEU-4 terbaik ada pada tercapai pada variasi dengan 2 layer dan hidden unit sebesar 256 dengan nilai BLEU-4 sebesar 0.0953 dan METEOR sebesar 0.2933. Ini merupakan *sweetspot* untuk model RNN.



Tetapi secara inkrementif, penambahan layer pada RNN berhubungan positif dengan performa model berdasarkan metrik BLEU-4 ataupun METEOR. Ini paling jelas terlihat dari variasi model dengan *hidden state* 128, di mana terdapat peningkatan yang konsisten dari penambahan layer. Di sisi lain, *hidden state* 256 terlihat penurunan drastis pada penambahan layer ketiga. Hal tersebut dapat terjadi karena model yang sudah cukup lebar lebih rentan mengalami *vanishing gradient* dan *overfitting* jika terlalu dalam.

4.2.2. Perbandingan Jumlah Layer: LSTM

Dengan serupa, grafik garis di bawah menunjukkan pengaruh jumlah layer terhadap performa model LSTM. Di kasus ini, diperoleh bahwa variasi dengan jumlah layer 2 dan *hidden unit* sebanyak 256 merupakan model terbaik jika dilihat dari BLEU-4 dengan nilai BLEU-4 sebesar 0.1181 dan METEOR sebesar 0.3157. Lalu, model LSTM sendiri tidak ada variasi yang sangat mendominasi (di kedua metrik sekaligus) seperti model RNN.

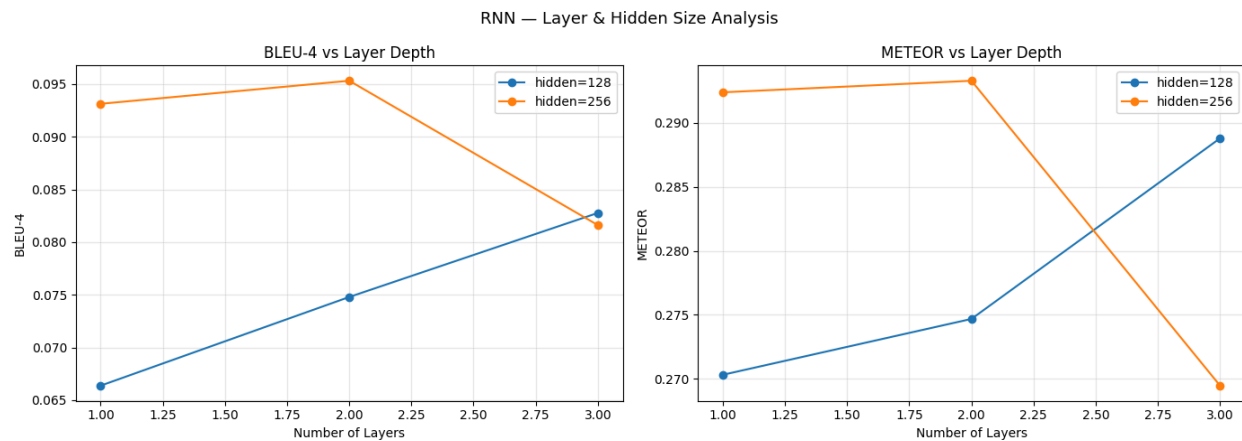


Kasus LSTM sendiri cukup menarik, karena berdasarkan skor BLEU-4, kurva untuk *hidden state* 256 serupa dengan RNN. Tetapi, tidak seperti pada RNN, variasi *hidden state* 128 malah menyerupai yang *hidden state* 256. Hal tersebut kemungkinan besar karena model LSTM

sendiri secara mendasar memiliki parameter yang lebih banyak dibandingkan RNN, sehingga risiko *overfitting*-nya lebih besar. Jadi, variasi seperti *hidden state* 128 dan 3 *layer* sudah cukup untuk membuat model *overfitting*. Untuk metrik METEOR, variasi *hidden state* 128 menunjukkan peningkatan konsisten, tetapi untuk *hidden state* 256 menunjukkan penurunan yang konsisten, yang lebih serupa dengan model RNN.

4.2.3. Perbandingan Ukuran Hidden State: RNN

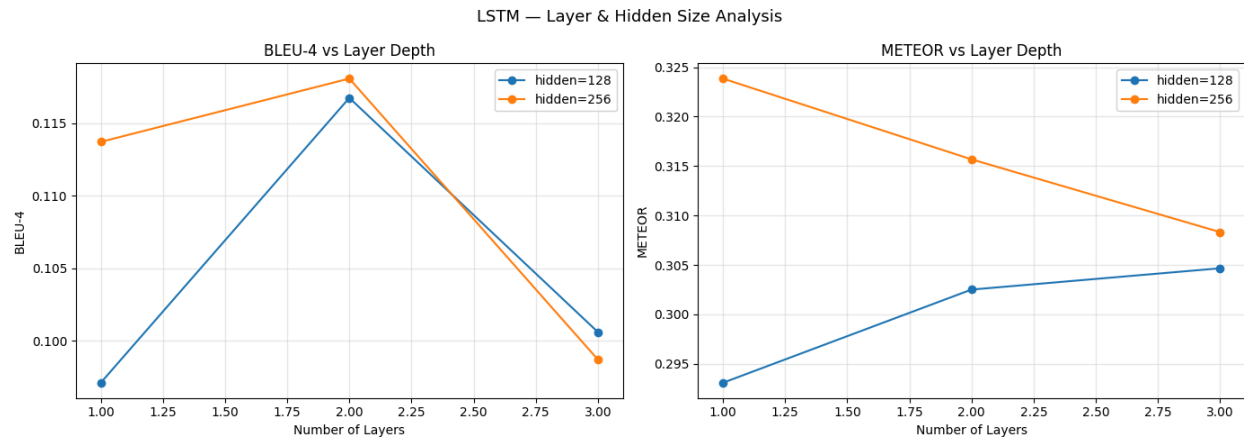
Agar memudahkan analisis, kami cantumkan kembali grafik garis dari 4.2.1 sebagai berikut. Sekarang, kami fokuskan diri pada perbedaan garis biru dan jingga.



Terlihat berdasarkan metrik BLEU-4 maupun METEOR bahwa penambahan hidden state memiliki dampak positif pada performa model. Bahkan, variasi terbaik dari kedua model RNN tercapai di jumlah *hidden state* terbesar. Namun dapat dilihat juga bahwa untuk konfigurasi *hidden state* 256, terjadi fenomena *overfitting* dan *vanishing gradient* ketika penambahan layer ketiga. Jadi, secara umum penambahan *hidden state* merupakan hal yang baik hingga batas tertentu, karena tetap harus seimbang dengan jumlah layer yang digunakan juga.

4.2.4. Perbandingan Ukuran Hidden State: LSTM

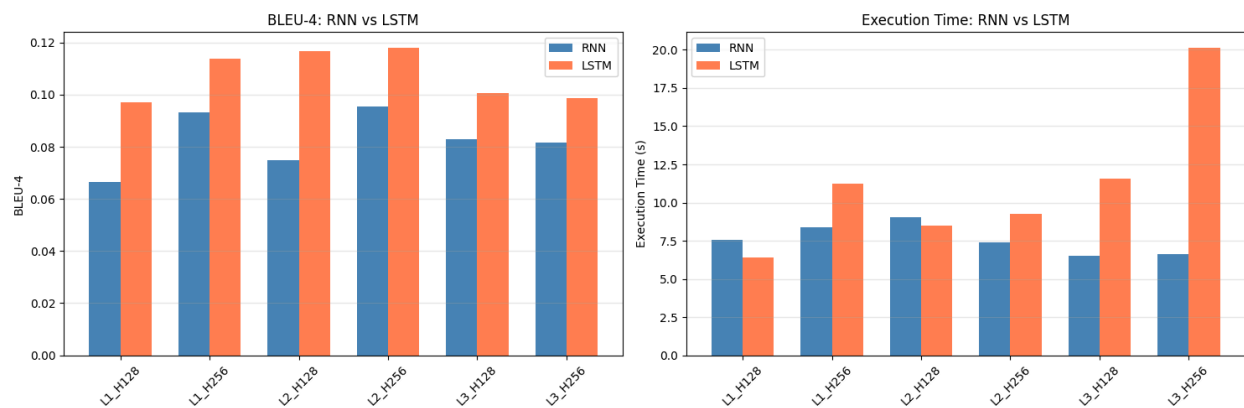
Lagi, agar memudahkan analisis, kami cantumkan kembali grafik garis dari 4.2.2 sebagai berikut. Kami fokuskan diri pada perbedaan garis biru dan jingga.



Pada LSTM, penambahan *hidden state* juga memberikan dampak positif serupa terhadap performa model terhadap metrik BLUE-4. Namun, pada metrik METEOR, terlihat bahwa penurunan yang konsisten pada *hidden state* 256, sedangkan peningkatan konsisten terjadi pada *hidden state* 128. Selebihnya, berdasarkan metrik METEOR, jumlah *hidden state* 256 sangat jelas lebih bermanfaat untuk performa terlepas jumlah layernya.

4.2.5. Perbandingan RNN vs LSTM

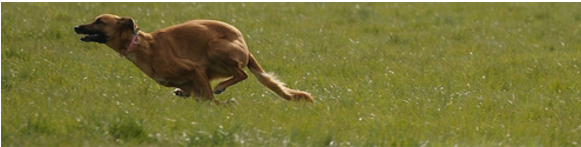
Sekarang, akan ditelaah lebih perbedaan dari RNN dan LSTM sebagai *decoder*. Grafik di bawah menggambarkan skor BLEU-4 dan waktu eksekusi untuk keenam variasi model.



Terlihat jelas pada grafik BLEU-4 bahwa LSTM mendominasi di setiap variasi arsitekturnya jika dibandingkan dengan RNN, tetapi execution timenya LSTM mayoritas lebih lama dibandingkan dengan RNN, terutama pada layer sebanyak 3 dan hidden unit sebesar 256. Keduanya memiliki alasan yang sama yaitu LSTM menggunakan 4 gates sedangkan RNN hanya 2 gates saja, sehingga dalam waktu eksekusi RNN akan jauh lebih cepat dibandingkan dengan LSTM, tetapi secara akurasi tentu akan kalah dari LSTM.

4.2.6. Analisis Kualitatif Image Captioning

Untuk *sanity check* dan mendapatkan pengertian lebih dalam tentang perbedaan RNN dan LSTM, berikut adalah beberapa sampel gambar dan bagaimana kedua model memprediksi caption-nya dibandingkan caption sebenarnya.

Images	Results
	<u>BLEU-4</u> : 0.904
	<u>RNN</u> : a black dog is running through the snow
	<u>LSTM</u> : a black dog is running through the wa
	<u>Target</u> : A black dog is running through the shallow edge of a large body of water
	<u>BLEU-4</u> : 0.869
	<u>RNN</u> : a dog is running through the grass
	<u>LSTM</u> : a brown dog is running through a field
	<u>Target</u> : A brown dog is running through a grassy field
	<u>BLEU-4</u> : 0.841
	<u>RNN</u> : a black dog is running through the snow
	<u>LSTM</u> : a black dog is running through the snow
	<u>Target</u> : A black dog is running through the snow



BLEU-4: 0.809

RNN: a dog is running through the snow

LSTM: a brown dog is running through the water

Target: A brown dog runs through the water as a wave comes



BLEU-4: 0.079

RNN: a man in a blue jacket is skiing down a snowy hill

LSTM: a man is skiing down a snowy hill

Target: A person climbing up a snowy mountain



BLEU-4: 0.079

RNN: a dog is running through the snow

LSTM: a dog is running through a grassy area

Target: A black dog is shaking off water



BLEU-4: 0.079

RNN: a man in a blue shirt is jumping on a rock

LSTM: a young boy is playing with a toy

Target: Children play with large hoops

	<u>BLEU-4</u> : 0.000
	<u>RNN</u> : a woman in a blue shirt and tie is standing on a sidewalk
	<u>LSTM</u> : a woman in a white dress is holding a baby in her hand
	<u>Target</u> : There are two ladies embracing and posing shown here
	<u>BLEU-4</u> : 0.000
	<u>RNN</u> : a person in a blue jacket is skiing down a snowy hill
	<u>LSTM</u> : a snowboarder is jumping over a snowy hill
	<u>Target</u> : The children make snow angles next to each other
	<u>BLEU-4</u> : 0.000
	<u>RNN</u> : a group of people are standing in front of a large building
	<u>LSTM</u> : a woman in a white dress is holding a baby in her hand
	<u>Target</u> : a group of children looking at something off-camera.

Mulai dari konteks dengan nilai BLEU-4 tinggi hingga rendah, semuanya konsisten bahwa model LSTM menunjukkan hasil yang lebih akurat dibandingkan RNN, hal tersebut disebabkan oleh LSTM yang memiliki kelebihan dalam "memahami" konteks dengan lebih baik karena komponen 4 gates yang dimilikinya. RNN yang tidak memiliki fitur tersebut dengan cepat mengalami vanishing gradient atau bisa dibilang "melupakan" konteks yang terjadi cukup jauh, sehingga hasil nya terbilang cukup halusinasi. Walaupun LSTM sudah cukup kuat untuk nilai BLEU tinggi, tetapi untuk konteks dengan BLEU-4 rendah, model LSTM pun cukup kesulitan untuk memahami dan memprediksi konteks yang sedang diberikan.

4.2.7. Perbandingan Keras vs Scratch

Berdasarkan eksperimentasi pengaruh jumlah layer dan ukuran, diperoleh bahwa variasi model yang paling optimal adalah varian ke-4 untuk kedua RNN dan LSTM.

Arsitektur	Layer	Hidden
RNN	2	256
LSTM	2	256

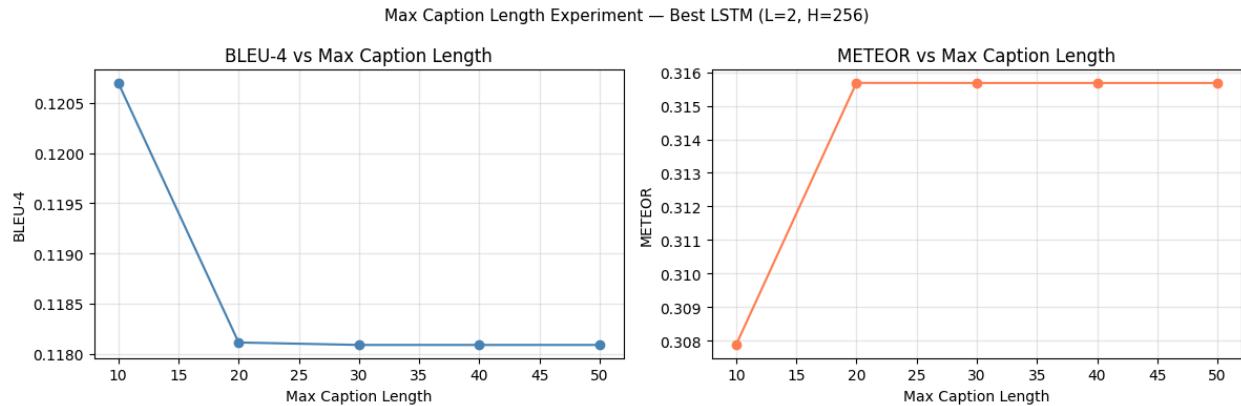
Jadi, menggunakan varian model ini, kami akan menguji kemampuan kedua package *from scratch* dan Keras dalam melakukan prediksi gambar (*forward pass*). Hasil perbandingan skor dan waktunya adalah sebagai berikut.

Package	BLEU-4	METEOR	Time (s)
RNN			
Scratch	0.0953	0.2933	7.4
Keras	0.0953	0.2933	58.2
LSTM			
Scratch	0.1181	0.3157	9.3
Keras	0.1181	0.3157	62.7

Untuk hasil dari scratch versus keras tentu akan menghasilkan hasil yang serupa, dimana hal tersebut menunjukkan bahwa implementasi logic dari kode from scratch telah sesuai. Tetapi seperti yang terlihat bahwa performance dari Scratch jauh lebih cepat dibandingkan dengan Keras, hal tersebut mungkin saja terjadi karena framework dari keras sendiri yang mengharuskan overhead ketika melakukan autoregressive decoding sehingga ada bottleneck disana.

4.2.8. Pengaruh Panjang Maksimum Caption

Berikutnya, menggunakan model terbaik LSTM yang sudah ditulis pada tabel di atas (2 layer dan 256 *hidden units*), kami menguji pengaruh panjang maksimum caption pada *tokenizer*. Grafik metrik BLEU-4 dan METEOR berdasarkan panjang maksimum tersebut di bawah.



Diperoleh bahwa untuk model terbaik LSTM ini, skor BLEU-4 memuncak pada *maximum length* 10. Hal tersebut mungkin saja terjadi karena pemotongan sekuens secara paksa membatasi model untuk hanya mengeluarkan kata-kata pembuka yang paling aman dan berakurasi tinggi, sekaligus mencegah munculnya kata-kata halusinasi di sisa kalimat. Sebaliknya, METEOR baru mencapai tingkat optimal pada *maximum length* 20. Ini mungkin karena sifat metriknya yang lebih mementingkan *recall* dan kelengkapan semantik dari deskripsi gambar. Lebih jauh lagi, skor dan waktu inferensi yang sepenuhnya stagnan dari batas 20 hingga 50 membuktikan bahwa arsitektur model telah berhasil memetakan token <end> dengan tepat sehingga proses dekode selalu dihentikan secara mandiri sebelum menyentuh batas karakter maksimum.

4.2.9. Perbandingan *Init-Inject*

Di sini, kami akan menguji penggunaan *init-inject* dibandingkan *pre-inject* (sesuai dengan arsitektur “Show and Tell” aslinya). Analisis ini akan dilakukan menggunakan model RNN dan LSTM dengan 1 layer dan *hidden state* sebanyak 256. Hasil performa model adalah sebagai berikut.

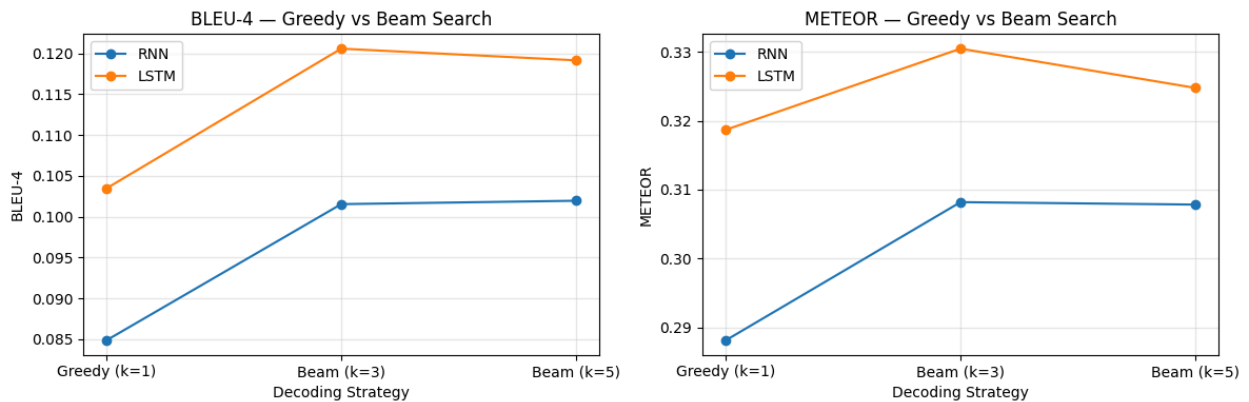
Metode	Model	Layer	Hidden	BLEU-4	METEOR	Time (s)
<i>Pre-Inject</i>	RNN	1	256	0.0931	0.2924	9.0s
<i>Pre-Inject</i>	LSTM	1	256	0.1137	0.3239	9.3s
<i>Init-Inject</i>	RNN	1	256	0.0970	0.3000	6.1s
<i>Init-Inject</i>	LSTM	1	256	0.0942	0.3024	8.0s

Hasil yang diperoleh pada kasus model RNN cukup unik, karena *init-inject* memperoleh hasil yang lebih bagus dibandingkan dengan *pre-inject*. Ini berbeda dari hipotesis teoritis kami: karena *pre-inject* menganggap vektor konteks seakan-akan itu kata pertama, "seharusnya" metode *pre-inject* akan lebih cepat mempertahankan konteks dibandingkan variasi *init-inject*,

yang menganggap vektor konteks hanya sebagai *hidden state*. Namun, perbedaan tersebut dapat disebabkan oleh vektor konteks yang terlalu sederhana, sehingga penggunaan *init-inject* akan memberikan konteks yang lebih lengkap dan RNN belum "lupa". Di sisi lain, kasus LSTM lebih selaras dengan teori kami. Untuk *execute time*-nya sendiri, tentu metode *init-inject* lebih cepat karena, vektor konteks langsung dimasukkan sebagai *hidden state* initial saja, sehingga tidak diperlukan sel tambahan untuk memproses vektor konteks seperti pada *pre-inject*.

4.2.10. Perbandingan *Beam Search*

Terakhir, kami menelaah *beam search* sebagai alternatif dari *greedy search* saat menerapkan komponen autoregresif dari *decoder*. Berikut adalah grafik yang menggambarkan metrik skor model berdasarkan metode *search* yang diterapkan.



Untuk melihat angka rincinya (serta waktu eksekusi), diberikan tabel berikut pula.

Model	Metode	BLEU-4	METEOR	Time (s)
RNN	Greedy	0.084873	0.288129	8.7
RNN	Beam ($k = 3$)	0.101533	0.308189	53.6
RNN	Beam ($k = 5$)	0.101949	0.307834	83.9
LSTM	Greedy	0.103468	0.318702	8.9
LSTM	Beam ($k = 3$)	0.120563	0.330481	75.6
LSTM	Beam ($k = 5$)	0.119133	0.324783	122.1

Berdasarkan grafik dan tabel, performa model yang menerapkan *beam search* sangat mendominasi terlepas hyperparameter k dan model RNN atau LSTM sekalipun. Hal tersebut karena *beam search* sendiri mencatat semua kandidat terbaik (*top k*), sehingga bersifat lebih *robust* dari *greedy search* yang hanya melihat satu kandidat terbaik. Namun, karena pencatatan

kandidat yang hati-hati tersebut, metode *beam search* jauh lebih lambat dibandingkan *greedy secara* secara waktu eksekusi. Maka, penggunaannya perlu menyeimbangkan efisiensi dan efektifitas yang diinginkan dari model. Selebihnya, peningkatan performa dari $k = 3$ ke $k = 5$ tidak terlalu signifikan, sehingga menunjukkan pemilihan k yang kecil sudah cukup untuk meningkatkan performa model tanpa mengorbankan waktu eksekusi.

BAB V

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Sebagai kesimpulan, kami mengimplementasikan suatu *package* neural network *from scratch*. Package yang dibuat mendukung banyak fitur-fitur yang membuatnya lengkap dan siap digunakan untuk pembangunan model *deep learning*.

1. *Package* bisa melakukan *forward* dan *backward pass* karena penggunaan data struktur kustom Tensor yang mendukung metode *auto-differentiation*. Berdasarkan pengujian, inferensi dan pelatihan juga sudah sesuai dengan hasil yang diperoleh dari Keras.
2. Terdapat implementasi lengkap fungsi aktivasi, metode inisialisasi bobot, fungsi loss, dan metode optimasi secara modular tanpa perlu memikirkan permasalahan gradien.
3. Diimplementasikan berbagai jenis layer, seperti dense, konvolusi, pooling, rekuren, embedding, dan lainnya. Setiap layer memiliki *hyperparameter* yang dapat diatur untuk mendapatkan hasil optimal.
4. Semua komponen dapat disatukan menjadi suatu model *deep learning* dengan kelas `Model` yang mendukung banyak API, seperti metode untuk pelatihan, prediksi, evaluasi, pemuatan bobot, penyimpanan bobot, dan lainnya.

Lalu, kami menerapkan *package* kami untuk memodelkan dua jenis permasalahan. Pertama, kami membangun suatu model *Convolutional Neural Network* (CNN) untuk melakukan klasifikasi gambar dataset [Intel Image Classification](#). Berdasarkan beberapa pengujian *hyperparameter* dan arsitektur yang dilakukan pada model CNN, diperoleh beberapa kesimpulan sebagai berikut.

1. Penambahan jumlah *layer* konvolusi dari 2 *layer* menjadi 3 *layer* memberikan peningkatan performa yang signifikan pada metrik evaluasi suatu model CNN. Akan tetapi, karena percobaan yang terbatas, akan terdapat suatu *threshold* ketika penambahan *layer* konvolusi tidak memberikan peningkatan performa model atau justru rentan terhadap *overfitting*.
2. Penambahan *filter* terbukti efektif meningkatkan kemampuan generalisasi model terhadap data yang belum pernah dilihat.
3. Penggunaan *filter* berukuran lebih kecil seperti ukuran 3x3, terbukti performa modelnya lebih efektif dan stabil dibandingkan *filter* berukuran lebih besar seperti 5x5.
4. Penggunaan *Max Pooling* secara umum memberikan performa yang lebih unggul dan stabil dibandingkan *Average Pooling*.

5. Penggunaan *parameter sharing* menyebabkan model lebih efisien secara komputasi dan tidak rentan terhadap *overfitting*, berbeda dengan jika menggunakan *non-parameter sharing* yang dapat menyebabkan fenomena ledakan jumlah parameter yang memicu *out-of-memory* dan *overfitting*.
6. *Library from scratch* telah diimplementasikan dengan sempurna dan akurat seperti *library from Keras*. Perbedaan utama terletak pada waktu komputasi yang dibutuhkan, dengan *library from Keras* lebih efisien dibandingkan *library from scratch*.

Kedua, kami membangun model *Encoder-Decoder* berbasis *Recurrent Neural Network* (RNN) dan *Long Short-Term Memory* (LSTM) untuk melakukan *image captioning* dataset [Flickr 8k](#). Berdasarkan beberapa pengujian *hyperparameter* dan arsitektur yang dilakukan pada model RNN, diperoleh beberapa kesimpulan sebagai berikut.

1. Penambahan jumlah layer rekuren cenderung merugikan, karena menambah risiko *vanishing gradient* serta *overfitting* secara signifikan.
2. Penambahan *hidden state* memberikan dampak positif pada layer sebanyak 1 dan 2, tetapi memiliki memberikan *diminish returns* pada layer sebanyak 3 pada kedua model.
3. Melalui analisis yang diberikan, disimpulkan LSTM memiliki keunggulan (bahkan mendominasi RNN) baik secara kuantitatif maupun kualitatif. Namun, LSTM kalah cukup dari segi *execute time* diandingkan RNN.
4. Hasil kedua metrik dari perbandingan *scratch* versus Keras menunjukkan hasil yang serupa. Namun, kode *scratch* jauh lebih cepat dari Keras, karena kelemahan framework yang dimiliki oleh Keras.
5. Pada model terbaik yaitu LSTM, nilai metrik BLEU-4 memuncak ketika *maximum length*=10, tetapi nilai METEOR memuncak disaat *maximum length* ≥ 20 .
6. Hasil *pre-inject* dari model RNN lebih rendah dibandingkan dengan *init-inject* nya, tetapi hasil *pre-inject* dari model LSTM lebih besar dibandingkan dengan *init-inject* nya.
7. Seluruh hasil *beam search* mendominasi hasil dari *greedy search*, karena memang keunggulan dari *beam search* yang dapat menyimpan dan mengevaluasi > 1 kandidat, namun keunggulan tersebut memiliki dampak yang meningkatkan *execute time* secara signifikan.

4.2. Saran

Berdasarkan hasil pengujian, analisis, dan kesimpulan yang telah dijabarkan, terdapat beberapa rekomendasi dan saran yang dapat diterapkan untuk pengembangan serta penelitian lebih lanjut pada masa mendatang.

1. Dapat dieksplorasi implementasi *forward pass* beberapa layer agar lebih efisien. Misalnya, layer konvolusi dapat menerapkan metode `im2col` atau transformasi Fourier. Lalu, Tensor dapat dibuat untuk mendukung penggunaan GPU.
2. Untuk menyelesaikan fenomena *overfitting* di kedua model CNN dan RNN, dapat dicoba beberapa metode seperti *data augmentation*, implementasi layer Dropout, atau layer Attention.
3. Agar menangkap performa model *Encoder-Decoder* dengan lebih baik, dapat diselidiki metrik lainnya seperti CIDEr dan ROUGE-L yang lebih menyerupai konsensus manusia.

DAFTAR PUSTAKA

- Fei-Fei, Li, Andrej Karpathy, and Justin Johnson. "Convolutional Neural Networks for Visual Recognition." *CS231n*, Stanford University, cs231n.github.io/convolutional-networks/. Accessed 9 May 2026.
- Karpathy, Andrej. "The spelled-out intro to neural networks and backpropagation: building micrograd." YouTube, 17 Aug. 2022. www.youtube.com/watch?v=VMj-3S1tku0.
- Kingma, Diederik P., and Jimmy Ba. (2014). "Adam: A Method for Stochastic Optimization." *arXiv*, arxiv.org/pdf/1412.6980.
- Selvaraju, Ramprasaath R. dkk. (2019). "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization". *arXiv*, arxiv.org/pdf/1610.02391.
- Tim Pengajar IF3270. (2026) "Convolutional Neural Network." *IF3270: Pembelajaran Mesin*, Institut Teknologi Bandung.
- Tim Pengajar IF3270. (2026) "Recurrent Neural Network." *IF3270: Pembelajaran Mesin*, Institut Teknologi Bandung.
- Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). "Show and Tell: A Neural Image Caption Generator". In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3156–3164, arxiv.org/pdf/1411.4555.

LAMPIRAN

A. Pembagian Tugas Kelompok

NIM	Nama	Pembagian Tugas
10122043	Beni Lesmana	pelatihan model RNN dan LSTM di Keras, implementasi arsitektur Encoder-Decoder, eksperimen dan evaluasi model Encoder-Decoder, dan bagian laporan yang sesuai
10122062	Albi Arrizkya Putra	pelatihan model CNN di Keras, implementasi arsitektur CNN, eksperimen dan evaluasi model CNN, dan bagian laporan yang sesuai
10123053	Timothy Niels Ruslim	implementasi utility functions, package utama yang termasuk forward dan backward propagation, layer-layer RNN dan CNN from <i>scratch</i> , feature extraction, preprocessing caption, dan bagian laporan yang sesuai

B. Tautan Penting

GitHub	github.com/timoruslim/Tubes2_AnakRudyKusdiantara
YouTube	—

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Beni Lesmana (10122043), Albi Arrizkya Putra (10122062),
Timothy Niels Ruslim (10123053)

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika (STEI)

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 2: Convolutional dan Recurrent Neural Networks

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: tidak berlaku.	Tidak	1
Pembuatan Teks: menggunakan tools seperti <i>Gemini</i> .	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti <i>Gemini</i> .	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: tidak berlaku.	Tidak	1
Pembuatan Kode: menggunakan tools seperti <i>Gemini</i> dan <i>Claude</i> .	Ya	3

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 12 Mei 2026



Timothy Niels Ruslim
10123053